



山东大学（青岛） · 计算机科学与技术学院

全国大学生计算机系统能力大赛

RespOS

操作系统内核设计文档

Rust · RISC-V 64 · LoongArch 64 · Linux ABI Compatibility

参赛队名	比特工匠队
队伍成员	李欣悦、肖安康、张俞睿
指导教师	颜廷坤、潘润宇
完成日期	2026 年 6 月

初赛设计文档

目 录

1 概述	5
1.1 项目介绍	5
1.2 代码结构	5
1.3 设计原则	6
1.4 整体架构	6
1.5 团队分工	7
1.6 初赛进展与排名 TODO	8
2 进程管理	9
2.1 概述	9
2.2 任务调度	9
2.3 调度队列与执行器	10
2.4 任务控制块	11
3 中断与异常处理	14
3.1 Trap 与特权级切换	14
3.2 Trap 处理流程	15
4 内存管理	17
4.1 地址空间总览	17
4.2 物理内存管理	19
4.3 进程内存管理	20
4.4 缺页异常处理	20
4.5 内核动态内存分配	22
4.6 用户地址检查	22
5 文件系统	23
5.1 虚拟文件系统	23
5.2 磁盘文件系统	24
5.3 非磁盘文件系统	25
5.4 页缓存	25
5.5 文件描述符表	26
6 进程间通信	27
6.1 信号机制	27
6.2 信号传输	27
6.3 信号处理	28
6.4 管道 (Pipe)	29
6.5 System V IPC 机制	30

6.6 Futex 同步	30
7 时钟模块	32
7.1 时钟源与时间尺度	32
7.2 周期中断与调度协作	33
7.3 用户可见时间接口	34
7.4 睡眠与超时	34
7.5 Interval Timer 与 POSIX Timer	35
7.6 timerfd	35
8 网络模块	37
8.1 Socket 套接字	37
8.2 回环网络设备	39
8.3 传输层: UDP 与 TCP	39
8.4 Port 端口分配	41
9 设备	43
9.1 设备管理模块概述	43
9.2 设备抽象与块设备	43
9.3 VirtIO 块设备	44
9.4 devfs 设备文件	45
9.5 控制台与平台设备配置	46
10 硬件抽象层	47
10.1 硬件抽象层总览	47
10.2 处理器访问接口	48
10.3 内核入口例程	49
10.4 内存管理单元与地址空间	50
11 总结与展望	54
11.1 工作总结	54
11.2 未来计划	55
12 AI 协作与开源项目借鉴	57
12.1 AI 辅助完成的工作	57
12.2 人工审核与边界控制	57
12.3 借鉴的开源项目	57
12.4 许可证与引用说明	57

文档约定

本文档默认采用 Linux / Unix 内核语境中的常见术语。模块名、系统调用名、结构体名、寄存器名和源码路径使用等宽字体；章节正文使用“任务”指代内核调度实体，使用“进程”指代线程组层面的用户可见语义。未特别说明时，虚拟地址、页表、文件描述符和信号语义均以 RespOS 初赛阶段实现为准。

缩写 / 术语	含义
ABI	应用二进制接口，本文主要指 Linux 用户态程序期望的系统调用和数据结构语义。
TCB	TaskControlBlock ，RespOS 中描述线程级调度实体的核心任务控制块。
TGID / TID	线程组 ID / 线程 ID；TGID 对应用户常见的进程 ID，TID 对应调度实体。
VMA	虚拟内存区域，对应 MapArea 记录的一段连续虚拟页区间。
COW	写时复制， fork 后共享只读页，首次写入时再复制物理页。
PTE / TLB	页表项 / 地址转换缓存，分别对应页表元数据和硬件地址翻译缓存。
VFS	虚拟文件系统抽象层，用统一 trait 连接 Ext4 、 procfs 、 devfs 、 pipe 等后端。
fd	文件描述符，用户态整数句柄，内核通过 FdTable 映射到 FileOp 对象。
trap	用户态因系统调用、中断或异常进入内核的统一控制流入口。

表 1 常用缩写与术语

第一章

概述

1.1 项目介绍

RespOS 是一个使用 Rust 编写的类 Unix 宏内核操作系统，面向操作系统能力大赛初赛测例和教学型内核演进场景。项目以 Linux ABI 兼容为主要目标，围绕进程管理、虚拟内存、文件系统、信号、IPC、时间、网络和设备驱动等基础能力展开，使 busybox、libc 初始化流程和 LTP 测例能够在 QEMU 虚拟平台中运行。

在硬件平台方面，RespOS 同时适配 RISC-V 64 与 LoongArch 64。两套架构在启动入口、页表机制、异常上下文和返回用户态流程上存在差异，因此项目将架构相关代码集中收敛到 arch 模块中，由上层任务、内存、文件系统和系统调用模块通过统一接口使用底层能力。这样的设计减少了上层模块对具体指令集的依赖，也为后续补齐双架构行为一致性留下清晰边界。

初赛阶段，RespOS 的实现重心不是单独堆叠系统调用入口，而是补齐用户程序运行链路中的关键基础设施。例如 `execve` 需要路径解析、ELF 装载、地址空间重建、用户栈构造和辅助向量；`fork/clone` 需要处理任务控制块、地址空间、文件描述符表和信号状态；文件相关测例则依赖 VFS、Ext4 后端、dentry cache、page cache 和 Linux 风格错误码共同工作。

目前项目仍处于持续开发阶段。现有内核已经形成较完整的模块边界，能够支撑团队继续围绕测例结果补齐细节语义、修复兼容性缺口并提升稳定性。

模块	当前设计重点
架构层	维护 RISC-V 64 与 LoongArch 64 的启动、trap、页表、上下文切换和平台接口。
任务管理	提供任务控制块、调度队列、内核栈、父子关系、fork/exec/wait 和 futex 等机制。
内存管理	实现页帧分配、内核高地址映射、用户地址空间、ELF 装载、mmap 和用户指针访问。
文件系统	构建 VFS、路径解析、文件描述符表、pipe、procfs/devfs、Ext4 后端和缓存层。
兼容接口	围绕 Linux ABI 实现系统调用分发、errno、信号、时间、IPC、网络回环和系统信息接口。

表 2 初赛阶段核心模块概览

1.2 代码结构

RespOS 仓库按照内核、用户程序、镜像脚本、测试工具和第三方依赖划分目录。内核主体位于 `os/`，用户态测试程序位于 `user/`，`vendor/` 保存当前阶段需要纳入构建的第三方代码，`judge/` 与 `scripts/` 用于测例统计、镜像处理和辅助构建。

```
RespOS/  
├─ Makefile                # 顶层构建入口
```

— bootloader/	# RustSBI 等启动固件
— docs/	# 开发记录、模块说明和本文档
— img/	# RISC-V / LoongArch 测试镜像
— judge/	# LTP 日志过滤、对比和报告脚本
— scripts/	# 镜像获取、测试统计等辅助脚本
— testsuit/	# 比赛测例相关材料
— user/	# 用户态程序、shell 和简单功能测试
— vendor/	# lwext4_rust、smoltcp、riscv 等依赖
— os/	# RespOS 内核源码
— Cargo.toml	
— build.rs	
— src/	
— arch/	# RISC-V 64 / LoongArch 64 架构相关代码
— drivers/	# VirtIO block 等设备驱动
— fs/	# VFS、Ext4、procfs、devfs、pipe、缓存
— mm/	# 页帧、堆、页表和地址空间管理
— net/	# socket、UDP/TCP、本地回环通信
— signal/	# 信号处理、sigaction、sigreturn 相关结构
— syscall/	# Linux ABI 系统调用分发与参数转换
— task/	# 任务控制块、调度器、内核栈、futex
— mutex/	# 自旋锁、睡眠锁和同步封装
— loader.rs	# 用户程序装载辅助
— main.rs	# 内核 Rust 入口与初始化流程

内核初始化入口在 `os/src/main.rs`。早期入口完成 BSS 清零和架构必要准备后进入 `rust_main_high`，随后依次初始化 trap、内存、网络、首个用户进程、定时器中断，最后进入任务调度循环。这个顺序反映了系统的基础依赖：任务创建依赖内存管理，用户程序运行依赖 trap 和地址空间，文件系统、网络 and 系统调用能力则在用户态负载执行过程中被逐步触发。

1.3 设计原则

RespOS 的实现围绕几个贯穿全文的原则展开。第一，采用宏内核结构，让任务、内存、文件系统和信号模块可以在一次系统调用内直接协作，优先降低初赛阶段补齐 Linux ABI 的工程成本。第二，使用 Rust 所有权和 RAII 管理资源生命周期：页帧由 `FrameTracker` 回收，任务、inode、dentry、FileOp 和共享内存页通过 `Arc/Weak` 表达共享关系。第三，保持调度策略简单，把复杂度留给阻塞、唤醒、退出和信号中断边界；当前 FIFO 队列不追求策略复杂度，而追求状态转换可解释。第四，用户地址不在内核中直接解引用，而是先检查 VMA 权限并主动处理懒分配或 COW，避免把普通系统调用错误退化不可控的内核缺页。

这些原则不是抽象口号，而是直接影响后续章节的结构：第二章的 TCB 聚合任务资源，第三章把 trap 返回作为信号递送点，第四章把地址合法性和页分配状态拆开，第五章用 VFS trait object 统一文件对象，第六章则让管道、共享内存和 futex 分别落在 fd、页帧和用户地址三个边界上。

1.4 整体架构

RespOS 采用宏内核结构，各核心模块运行在同一内核地址空间中。初赛阶段选择这一结构，主要是为了降低跨模块协作成本：`execve`、`fork`、`mmap`、`pipe`、`signal` 和路径解析等接口都不是单模块功能，需要任务、内存、文件系统、信号和系统调用层共同维护 Linux ABI 语义。

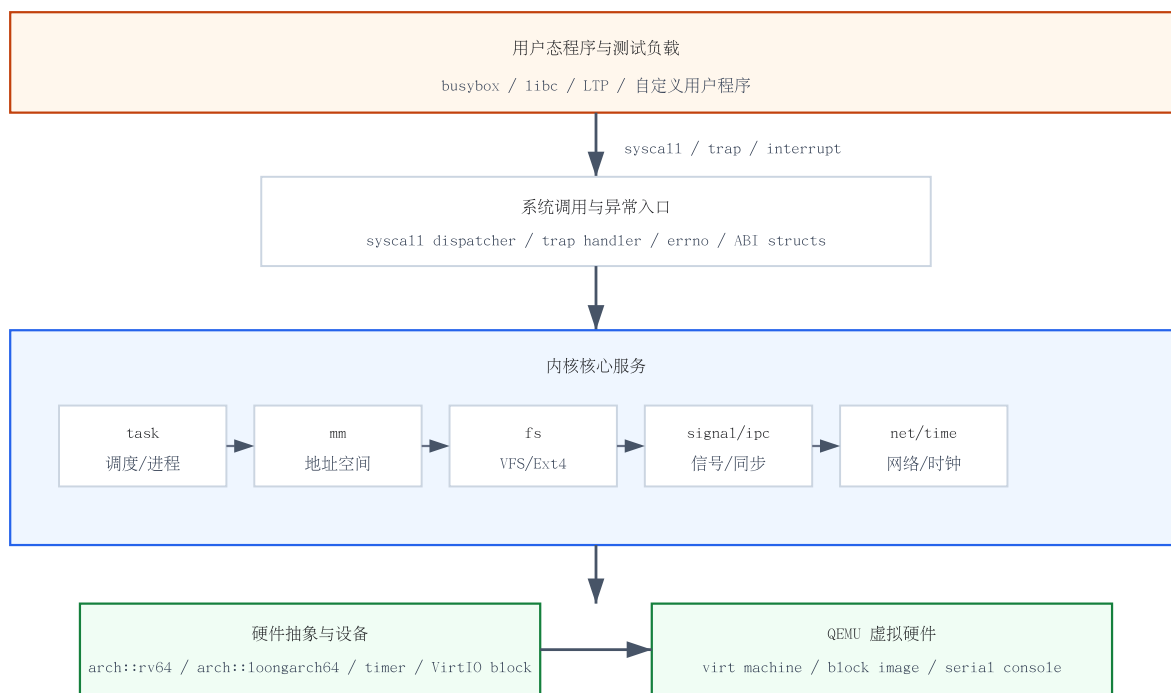


图 1 RespOS 整体架构示意

这种结构的优点是开发路径直接，便于在测例驱动下快速补齐能力；代价是模块之间存在真实的语义耦合。因此项目需要通过清晰的接口约定和回归测试约束资源生命周期，避免文件描述符、地址空间、信号状态和父子关系在复杂系统调用路径中出现不一致。

以用户执行 `cat /proc/cpuinfo` 为例，shell 先通过 `fork` 创建子任务，子任务执行 `execve` 装载 `cat`；系统调用层进入文件系统后，VFS 从当前 `cwd/root` 出发解析 `/proc/cpuinfo`，挂载树把路径切换到 `procfs` 后端；`procfs` 的 `inode` 动态生成 CPU 信息文本，常规 `read` 再通过 `fd` 表中的 `FileOp` 返回给用户缓冲区。这个短路径同时穿过任务、trap、用户地址检查、VFS、`procfs` 和 `fd` 表，正是 RespOS 选择宏内核和清晰模块边界的原因。

1.5 团队分工

当前项目由三名队员协作推进，张俞睿担任队长并承担项目主要开发工作。由于内核模块之间耦合较高，实际开发中会根据测例压力和 bug 所在模块动态交叉支持；下表描述的是初赛阶段的主要责任边界。

成员	主要工作
张俞睿（队长）	负责项目总体设计和主要代码实现，推进内核主体开发、双架构适配、任务/内存/文件系统/系统调用等核心模块联调，并维护构建脚本、测例辅助工具和阶段性问题定位。
李欣悦	负责文档统筹、测试记录整理、系统调用兼容性梳理以及部分模块联调，协助维护初赛测例运行流程。
肖安康	参与内核功能开发与调试，协助任务管理、内存管理、异常处理、文件系统与 Linux ABI 兼容相关实现。

表 3 团队分工

在协作方式上，团队优先围绕“能否运行用户态负载”和“测例失败原因是否可定位”推进工作。对于影响多个模块的功能，会先明确系统调用语义、涉及的数据结构和错误码边界，再进入具体实现；对于不稳定测例，会保留日志、复现步骤和阶段性处理方案，避免临时修补分散在难以审查的位置。

1.6 初赛进展与排名 TODO

截至本文档当前版本，RespOS 仍处于初赛功能补齐和稳定性改进阶段。项目已经建立双架构源码结构，并围绕文件系统、任务管理、内存管理、信号、IPC、网络回环和设备驱动形成了可继续迭代的内核主体。后续将继续以初赛测例结果为依据，补齐 Linux ABI 的边界行为和错误路径。

TODO：初赛排名图

当前比赛仍在开发和测试过程中，最终排名截图、测例通过情况和阶段性结果图将在初赛结果稳定后补充到本节。

本章小结

概述章给出 RespOS 的目标、代码组织和团队责任边界。后续章节将沿着用户程序运行所依赖的主路径展开：任务被调度，trap 进入内核，地址空间提供隔离，文件系统和其他内核服务再向 Linux ABI 补齐具体语义。

第二章

进程管理

2.1 概述

进程管理模块负责把用户程序抽象为可调度、可等待、可复制、可替换的内核对象。RespOS 将“任务”作为调度基本单位，用 `TaskControlBlock` 描述一个线程级执行实体；多个任务可以通过 `ThreadGroup` 组成 Linux 风格线程组，此时线程组 ID (TGID) 对应传统意义上的进程 ID，而每个任务仍拥有独立的线程 ID (TID)。

从实现角度看，进程管理承担多个模块的汇合点角色。创建任务需要内存模块提供地址空间和内核栈映射；执行 `execve` 需要文件系统提供可执行文件数据，并由内存模块重建用户地址空间；`wait4` 需要维护父子关系和退出状态；线程退出还会与 `futex`、信号和 `clear_child_tid` 等用户态同步机制协作。因此本章重点描述 RespOS 如何组织任务对象、如何调度任务，以及如何在 Linux ABI 语义下区分进程和线程。

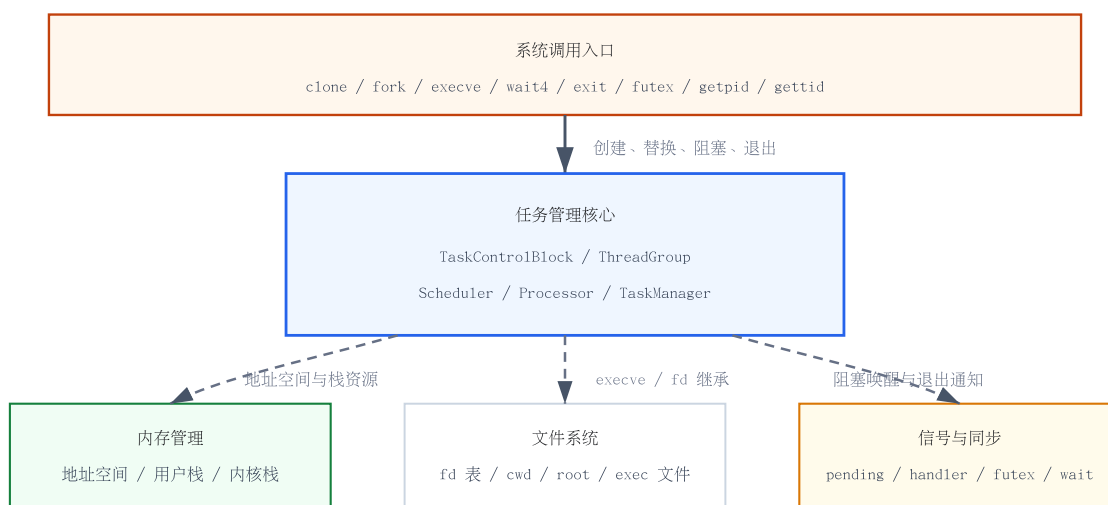


图 2 进程管理模块在内核中的位置

进程管理模块的主要源文件位于 `os/src/task/`。其中 `task.rs` 定义任务控制块和进程/线程资源，`scheduler.rs` 维护就绪队列与阻塞队列，`processor.rs` 记录当前 CPU 正在运行的任务，`manager.rs` 提供 TID 到任务对象的全局弱引用索引，`context.rs` 和 `kstack.rs` 则分别负责任务上下文和内核栈布局。

2.2 任务调度

RespOS 当前采用 FIFO 调度策略。就绪任务被放入 `ready_queue`，调度器每次从队首取出下一个任务执行；当任务主动让出 CPU、阻塞等待事件、收到停止信号或退出时，调度路径会更新任务状

态，并通过架构层 `__switch` 切换到下一个任务的内核栈和地址空间。`__switch` 只负责内核上下文切换，真正返回用户态还要依赖第三章描述的 `__restore`。

任务首次运行时，内核在任务内核栈上放置 `TrapContext` 和 `TaskContext`。`TaskContext` 的返回地址指向 `__restore`，因此任务被调度后会进入异常返回流程，再恢复到用户态入口。后续任务切换时，`__switch` 保存当前任务的内核上下文，恢复下一个任务的内核栈指针、返回地址和页表 token，使任务能够从之前暂停的位置继续执行。

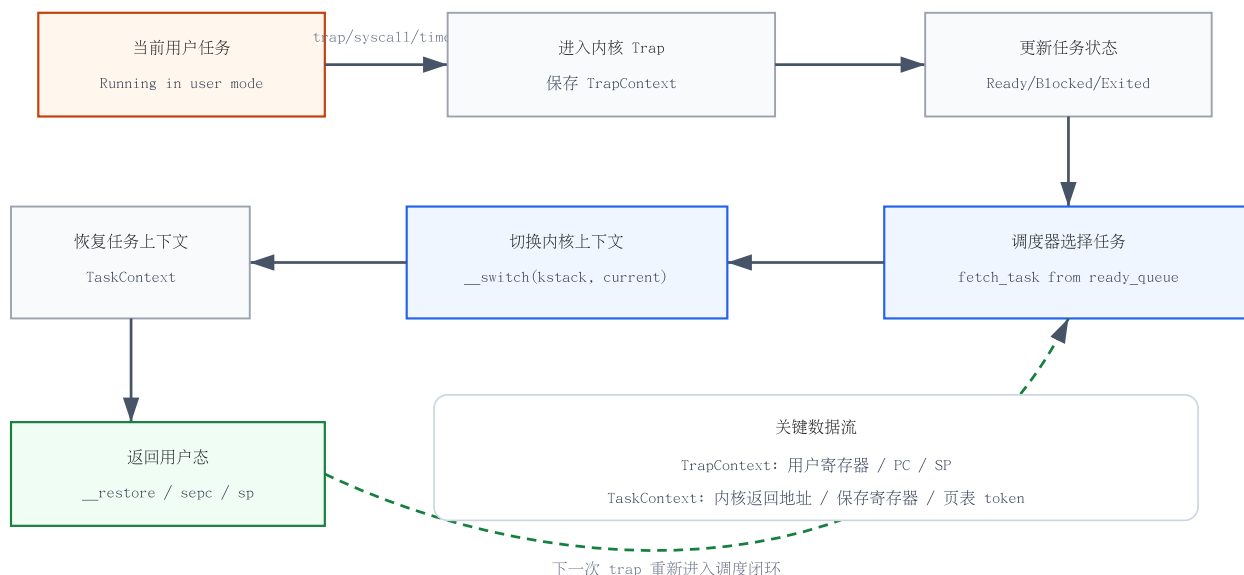


图 3 任务切换流程

调度器目前不引入复杂优先级、时间片权重或多核心负载均衡，主要目标是保证初赛阶段系统调用和用户态程序运行链路稳定。对于 `busybox`、`libc` 初始化和 LTP 测例而言，调度策略本身不是瓶颈，关键在于阻塞、唤醒、退出和父子回收路径必须语义清楚，不能丢失任务状态或错误释放资源。

2.3 调度队列与执行器

调度器由两个核心队列组成：`ready_queue` 保存可运行任务，`blocked_queue` 保存因等待事件而暂时不可运行的任务。`add_task` 会将就绪任务加入队尾，`fetch_task` 从队首取出任务，`block_task` 将阻塞任务放入阻塞队列，`wakeup_task` 则根据 TID 将任务从阻塞队列移回就绪队列。

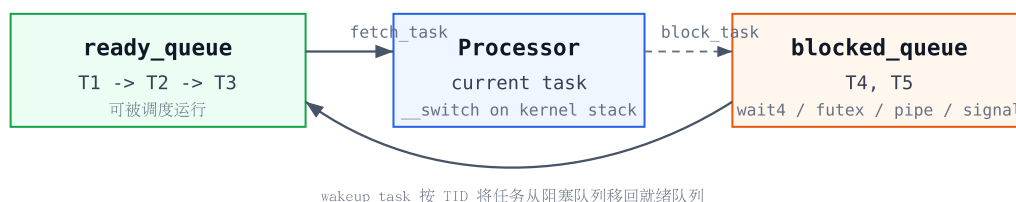


图 4 调度队列结构

`processor.rs` 中的 `PROCESSOR` 保存当前 CPU 正在运行的任务。系统启动后，`run_tasks` 从就绪队列取出第一个任务，将其记录为当前任务，再切换到该任务内核栈。此后常规调度不再回到独立的执行器对象，而是由当前任务在让出、阻塞或退出路径中直接选择下一个任务并调用 `__switch`。

为了便于按 TID 查找任务，`TaskManager` 维护一个 `HashMap<tid, Weak<TaskControlBlock>>`。调度队列负责“谁接下来运行”，任务管理器负责“能否通过 TID 找到任务对象”，二者职责不同。这样的拆分让 `kill`、`futex wake`、`wait` 等路径可以按 TID 找到目标任务，而不会强依赖任务是否仍在 `ready` 队列中。

结构	职责
<code>Scheduler</code>	维护 FIFO 就绪队列和阻塞队列，负责添加、取出、阻塞、唤醒和移除任务。
<code>Processor</code>	记录当前 CPU 正在运行的任务，为 <code>current_task</code> 和上下文切换提供当前任务引用。
<code>TaskManager</code>	维护 TID 到任务对象的全局弱引用索引，支持按 TID 查询和遍历任务。
<code>KernelStack</code>	为每个任务分配内核栈 <code>slot</code> ，并在任务生命周期内维护栈顶位置。
<code>TaskContext</code>	保存任务切换所需的返回地址、被调用者保存寄存器和页表 <code>token</code> 。

表 4 调度相关结构职责

2.4 任务控制块

`TaskControlBlock` 是任务管理模块最核心的数据结构，用于把一个可调度实体的身份、运行上下文和进程资源聚合到同一个生命周期中。为了支持中断、系统调用和唤醒路径中的并发访问，内部可变量段通常通过 `SpinLock`、`RwLock` 或原子变量保护；具体字段按职责分组如下。

身份与关系	运行上下文	进程资源
<code>tid / tgid / pgid / sid</code>	<code>kernel_stack / task_status</code>	<code>fd_table / cwd / root</code>
<code>parent / children / exit_code</code>	<code>memory_set / TaskContext</code>	<code>exe_path / limits</code>
信号资源	线程同步	计时统计
<code>sig_pending / sig_stack</code>	<code>clear_child_tid</code>	<code>itimers / start_time</code>
<code>sig_handler</code>	<code>robust_list / futex</code>	<code>child_utime / child_stime</code>

表 5 `TaskControlBlock` 结构分组

任务创建时会先分配 TID 和内核栈，再构造用户地址空间，并在内核栈顶部布置 `TrapContext` 与 `TaskContext`。`execve` 不创建新的任务对象，而是在当前任务上替换地址空间、重建用户栈、关闭 `close-on-exec` 文件描述符，并重置信号处理状态。

```
pub struct TaskControlBlock {
    kernel_stack: KernelStack,
    tid: RwLock<TidHandle>,

```

```
memory_set: Arc<RwLock<MemorySet>>,
fd_table: SpinLock<Arc<FdTable>>,
sig_pending: SpinLock<SigPending>,
}
```

2.4.1 进程与线程关系

RespOS 采用接近 Linux 的“线程是调度单位，进程是线程组”的模型。每个 `TaskControlBlock` 都是一个可调度任务，拥有唯一 TID；当多个任务属于同一个 `ThreadGroup` 时，它们共享 TGID，用户态通常把 TGID 视为进程 ID。 `gettid` 返回当前任务的 TID， `getpid` 返回当前线程组的 TGID。

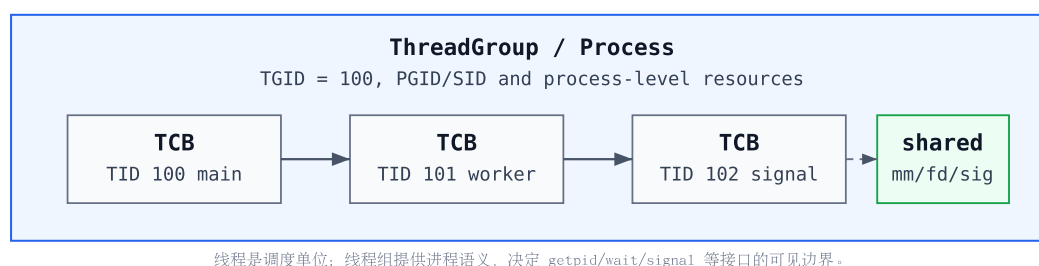


图 5 进程与线程关系

`clone` 标志决定新任务是线程还是进程。当设置 `CLONE_THREAD` 时，新任务加入调用者的线程组，并共享地址空间、部分文件系统上下文和信号处理函数表；未设置该标志时，新任务成为独立进程，拥有新的 TGID，并作为子进程加入父任务的 `children` 表。这个区分直接影响 `wait4` 回收语义：普通子进程由父进程等待回收，同线程组内的新线程不进入父进程的 `children` 表。

这种模型使 RespOS 能够兼容 `libc` 和 `pthread` 中常见的线程创建方式，同时保留传统 `fork/exec/wait` 进程语义。对于初赛测例而言，正确处理 TID、TGID、父子关系和线程组退出，是 `shell`、`busybox` 和 `LTP` 进程类测例能否稳定运行的关键。

2.4.2 任务状态

任务状态由 `TaskStatus` 表示，它不只是调度器内部标记，也会影响 `wait4`、信号处理、`futex` 唤醒和资源释放。状态变化必须与队列操作保持一致，否则容易出现任务已经阻塞但仍被调度、任务已经退出但无法被父进程回收等错误。

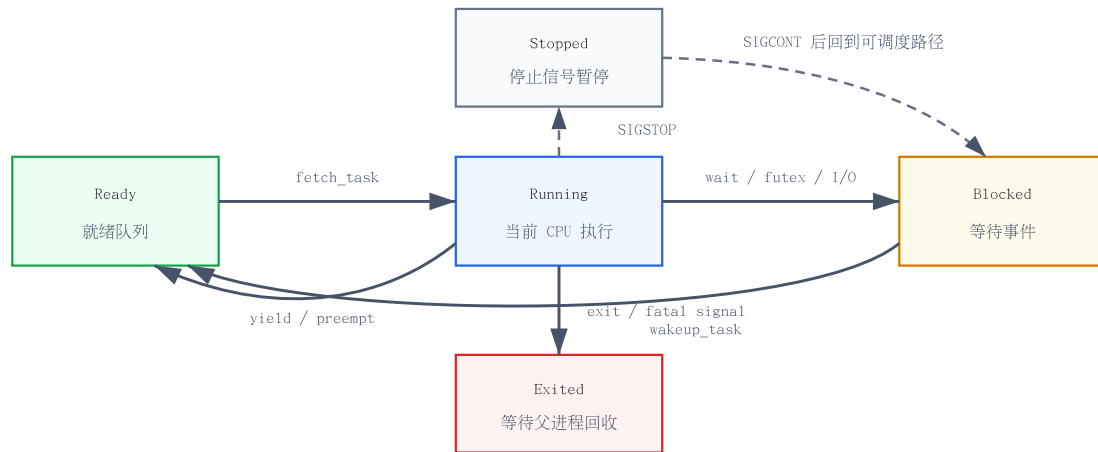


图 6 任务状态转换

其中 **Exited** 不是立即释放所有资源的终点。普通子进程退出后仍需要保留退出码和必要的任务元数据，直到父进程通过 `wait4` 完成观察和回收；线程退出还需要配合 `futex`、`clear_child_tid` 和线程组状态完成用户态同步。

本章小结

进程管理章的关键设计是把调度单位、线程组语义和进程资源生命周期分开处理。这样 `clone`、`fork`、`execve`、`wait4`、信号和 `futex` 可以共享同一套任务对象，同时仍能保留 Linux 对 TID、TGID、父子回收和文件偏移共享的预期。

第三章

中断与异常处理

3.1 Trap 与特权级切换

中断与异常处理模块负责把用户态执行流安全转入内核，并在处理完成后恢复用户态上下文。RespOS 在 RISC-V 64 和 LoongArch 64 上分别实现架构相关入口，但上层处理抽象保持一致。整体流程是入口汇编保存 `TrapContext`，Rust 侧 `trap_handler` 完成异常分发，最后通过 `__restore` 返回用户态或进入调度路径。

从用户态进入内核时，硬件只保存最小异常现场并跳转到架构指定入口。两套架构都不会自动保存所有通用寄存器，因此 RespOS 在入口汇编中显式构造完整 `TrapContext`。

```
__trap_from_user:
    csrrw sp, sscratch, sp
    addi sp, sp, -64*8
    sd x1, 1*8(sp)
    sd x3, 3*8(sp)
    csrr t0, sstatus
    csrr t1, sepc
```

项目	RISC-V 64	LoongArch 64
异常入口	<code>stvec</code> 指向 <code>__trap_from_user</code> / <code>__trap_from_kernel</code>	<code>EENTRY</code> 指向用户或内核 <code>trap</code> 入口
返回指令	<code>sret</code>	<code>ertn</code>
返回地址	<code>sepc</code>	<code>ERA</code>
异常原因	<code>scause</code>	<code>ESTAT</code>
坏地址	<code>stval</code>	<code>BADV</code>
用户/内核栈交换	<code>sscratch</code> 保存用户栈和内核栈切换信息	<code>KSAVE0</code> 保存用户栈和下次入口内核栈顶

表 6 双架构 trap 关键信息对照

`TrapContext` 的设计承担两个职责：一是保存用户态被打断时的通用寄存器、返回地址和特权状态；二是作为系统调用参数和返回值的传递载体。RISC-V 使用 `a7/x17` 作为系统调用号、`a0-a5/x10-x15` 作为参数并把返回值写回 `a0`；LoongArch 使用 `a7/r11` 作为系统调用号、`a0-a5/r4-r9` 作为参数并把返回值写回 `a0/r4`。因此系统调用分发层可以使用统一的 `syscall(id, args)` 接口，而寄存器细节被收敛在各架构 `TrapContext` 实现中。

3.2 Trap 处理流程

RespOS 的 trap 处理分为入口保存、Rust 分发、后置检查和返回恢复四个阶段。入口汇编只做与架构强相关的工作，包括保存寄存器、切换异常入口、准备 `&mut TrapContext` 参数；具体语义由 Rust 侧完成，这样系统调用、缺页、信号和定时器逻辑可以尽量复用上层模块。

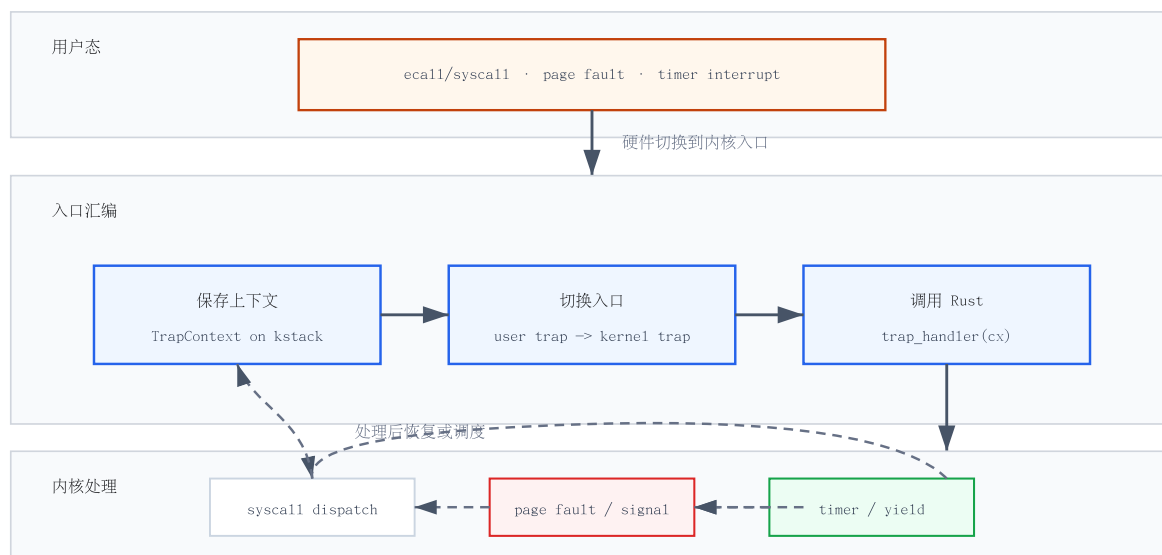


图 7 用户态 trap 处理流程

图中各分支的处理结果并不相同：有的修改上下文后继续返回，有的先尝试修复，失败后转为信号或退出，有的直接进入调度。具体边界在用户态 trap 表中列出。

3.2.1 内核态异常处理

内核态异常和用户态异常采用不同入口。用户态进入 trap 后，入口汇编会把后续异常入口切到内核 trap 路径；返回用户态前再切回用户 trap 路径。这样如果内核在处理系统调用或缺页时再次触发异常，不会错误地按用户态现场保存方式处理。

当前内核态 trap 采用保守处理策略：断点异常可以打印信息并跳过断点指令；非法指令、内核缺页、内核态系统调用等情况直接 panic。原因是这些异常通常代表内核 bug 或关键假设被破坏，继续执行可能扩大破坏范围。定时器中断若出现在内核态，目前主要记录日志，不在内核路径中执行复杂调度。

3.2.2 用户态中断与异常处理

用户态 trap 是系统调用、缺页、非法指令、断点和定时器中断的统一入口。对于可恢复事件，内核会修改 `TrapContext` 后返回用户态；对于不可恢复事件，内核会向任务递送信号或直接终止任务。不同 trap 类型的修复动作和最终状态如下。

trap 类型	处理模块	修复动作	最终状态
系统调用	syscall	跳过 syscall 指令，分发到具体实现，写回返回值或错误码。	返回用户态; sigreturn 可直接恢复旧上下文。
缺页异常	mm::MemorySet	根据访问类型尝试懒分配、写时复制、权限检查和 VMA 修复。	修复成功后返回；失败时递送 SIGSEGV 或结束任务。
定时器中断	timer / task	设置下一次触发，检查任务定时器和 POSIX timer。	当前任务让出 CPU，进入调度路径。
非法指令	task	记录异常现场。	结束当前任务。
断点异常	task	记录断点位置。	结束当前任务。

表 7 用户态 trap 类型与处理结果

在用户态 trap 返回前，内核还会统一检查任务定时器和待处理信号。这样做的好处是信号递送点比较集中：系统调用返回、缺页修复后返回、定时器触发后的调度出口都可以进入同一套 `handle_signals` 逻辑。对于 libc、pthread 和 LTP 测例来说，信号能否在阻塞等待、定时器到期和系统调用返回边界被及时观察，是兼容性的重要部分。

3.2.3 返回用户态

返回用户态由 `__restore` 完成。该路径从当前任务内核栈上的 `TrapContext` 恢复通用寄存器、返回地址和特权状态，并将异常入口重新设置为用户 trap 保存路径。RISC-V 在恢复 `sstatus` 与 `sepc` 后执行 `sret`；LoongArch 在恢复 `PRMD` 与 `ERA` 后执行 `ertn`。从用户程序视角看，除了系统调用返回值、信号处理上下文或异常导致的退出外，普通 trap 应当像一次透明的控制流暂停。

返回路径还承担下一次 trap 的准备工作。RISC-V 使用 `sscratch` 保存用户栈和内核栈切换所需的信息；LoongArch 在返回前把内核栈顶写入 `KSAVE0`，并把内核 `tp` 保存到 `KSAVE1`，保证下一次用户态异常进入内核后可以恢复正确的内核线程局部状态。

`__restore` 也是调度路径和 trap 路径的交汇点。首次运行任务时，调度器恢复的 `TaskContext` 会跳到 `__restore`，由它把预先布置在内核栈上的 `TrapContext` 恢复成用户态入口；普通系统调用返回时，`trap_handler` 修改同一份 `TrapContext` 后也回到 `__restore`。因此这里必须同时保证当前返回正确、下一次异常入口正确、架构特权状态正确，任何一个细节出错都会表现为用户栈损坏、重复执行系统调用或无法从信号处理返回。

本章小结

中断与异常处理章的重点是把双架构入口差异收敛到统一的 `TrapContext` 和 Rust 分发逻辑中。系统调用、缺页、定时器和信号递送最终都通过 trap 返回路径闭合，因此 `__restore` 同时是异常处理、调度和用户态恢复的共同边界。

第四章

内存管理

4.1 地址空间总览

内存管理模块负责维护内核和用户进程的虚拟地址空间，为任务创建、`execve`、`fork`、`mmap` 和用户缓冲区访问等路径提供基础能力。RespOS 将地址空间抽象为 `MemorySet`，其中包含根页表、逻辑段列表、堆边界和 `mmap` 分配游标；每个逻辑段由 `MapArea` 表示，描述一段连续虚拟页的映射类型、访问权限和已经分配的物理页帧。

在双架构支持上，RISC-V 64 和 LoongArch 64 都采用 4 KiB 页和 39 位虚拟地址宽度，内核高地址线性映射、用户低地址空间、`mmap` 区间和 `sigreturn` 跳板页保持同一套常量布局。架构差异集中在页表项编码、页表根写入方式和 TLB 刷新指令上，上层 `MemorySet` 通过统一的 `PageTable`、`PageTableEntry` 和 `MapPermission` 接口使用这些能力。

项目	RISC-V 64	LoongArch 64
页表 token	<code>satp = MODE(Sv39) root_ppn</code>	<code>root_ppn << 12</code> ，写入页表根 CSR
页表根寄存器	<code>satp</code>	<code>PGDL</code> 和 <code>PGDH</code> 同时写入当前 <code>root</code>
TLB 刷新	<code>sfence.vma</code>	<code>invtlb</code> 刷新全局和 ASID 相关项
PTE 软件位	使用 <code>RSW</code> 位保存 <code>COW</code> 标志	使用软件位保存 <code>COW</code> 标志，并转换为 LoongArch PTE 编码

表 8 双架构页表机制对照

4.1.1 地址空间布局

RespOS 采用“用户低地址、内核高地址”的布局。用户进程的页表并不是从零开始构造完整内核映射，而是通过 `PageTable::from_kernel` 复制内核高地址部分的根页表项，使每个用户地址空间都能在陷入内核后继续访问内核代码、数据、堆、设备映射和物理页线性映射。

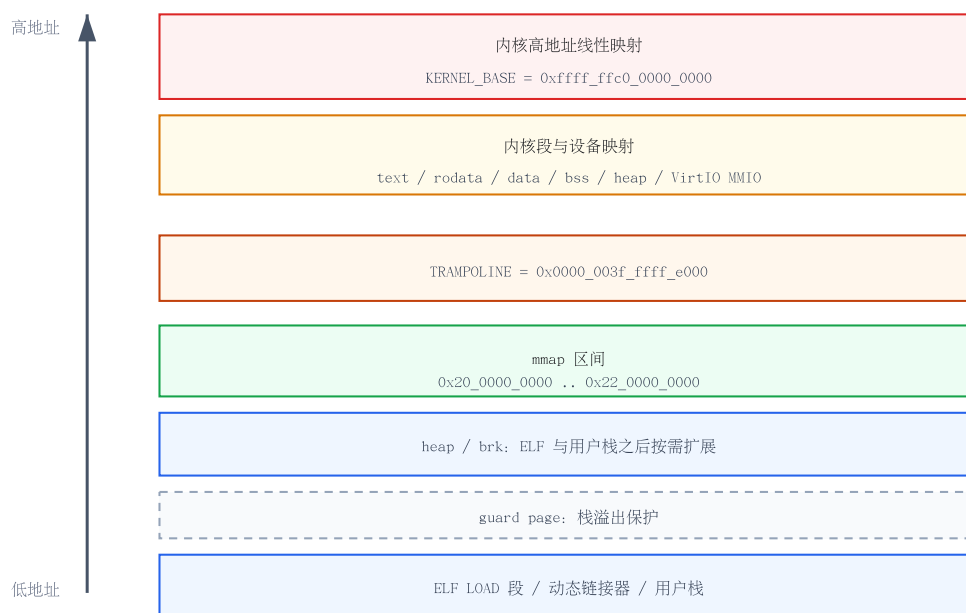


图 8 RespOS 地址空间布局

用户地址空间由 ELF LOAD 段、动态链接器映射、用户栈、堆区、mmap 区间和 sigreturn 跳板页组成。加载 ELF 时，内核根据程序头权限建立 READ、WRITE、EXECUTE、USER 标志；最高 LOAD 段之后先留出一页 guard page，再放置用户栈，栈顶位于栈区域高地址端，向低地址增长时会先撞到 guard page。heap_bottom 和初始 brk 放在用户栈之后，堆通过 brk 向高地址扩展；mmap 区间从 MMAP_MIN_ADDR 独立开始，和早期堆/栈之间保留较大空洞，避免普通堆增长立即碰到 mmap 区。

4.1.2 地址翻译模式

地址翻译先把虚拟地址按 4 KiB 页大小拆成 VPN 和页内偏移，再用 VPN 的三级索引沿根页表逐级查找页表项。中间级页表项不存在时，find_pte_create 会分配新的页表页；叶子页表项有效时，内核取出 PPN，并把 PPN 对应物理页基址加上原始页内偏移，得到最终物理地址。

VirtAddr、VirtPageNum、PhysAddr 和 PhysPageNum 负责地址与页号转换；PageTable 负责沿三级页表查找或创建页表项；MapArea 决定虚拟页最终映射到哪类物理页。内核直接映射区使用 MapType::Direct，物理页号可以由虚拟页号减去 KERNEL_BASE 对应页号得到；用户空间和 mmap 区域使用 MapType::Framed，由页帧分配器按需分配物理页。

结构	职责
MemorySet	维护一个地址空间的根页表、逻辑段、堆边界和 mmap 游标。
MapArea	描述连续虚拟页区间的映射类型、权限、共享属性和已分配页帧。
PageTable	创建三级页表、建立/取消映射、修改 PTE 标志并生成架构 token。
FrameTracker	持有一个物理页帧，离开生命周期后自动归还页帧分配器。

表 9 内存管理核心结构职责

4.1.3 启动阶段预映射

内核启动后会先初始化堆分配器和物理页帧分配器，再创建 `KERNEL_SPACE`。内核地址空间按照链接脚本导出的段边界分别映射 `.text`、`.rodata`、`.data`、`.bss` 与初始栈，并把 `ekernel` 之后到物理内存结束的区域作为可读写线性映射加入页表。设备 MMIO 区域同样以 `MapType::Direct` 方式映射到内核高地址，供 `VirtIO` 等驱动访问。

这部分映射必须在用户任务创建之前完成，因为用户页表的内核高地址部分会从 `KERNEL_SPACE` 派生。如果内核段权限配置错误，结果通常不是单个用户进程崩溃，而是 `trap`、调度、文件系统或设备访问路径整体不可用；因此内核段按照最小权限区分可执行、只读和可写区域。

需要注意的是，线性映射和页帧分配不是同一件事。线性映射只是让内核能够通过 `KERNEL_BASE + pa` 访问这段物理内存；某个物理页是否已经被占用，仍然由页帧分配器及其 `FrameTracker` 所有权决定。因此 `ekernel` 之后的物理区间既可以被高地址线性访问，也可以作为空闲页帧池的一部分被逐页分配。

4.1.4 内核地址转换

物理页内容通过 `PhysPageNum::get_bytes_array`、`get_pte_array` 和 `get_mut` 访问。RISC-V 路径始终将物理地址加上 `KERNEL_BASE` 得到内核虚拟地址；LoongArch 路径在分页开启后采用同样的高地址线性映射，分页开启前则保留直接物理访问能力。这样页表页、用户页和文件页都可以通过统一的物理页接口读写，而不需要临时把用户虚拟页映射到内核地址空间。

4.2 物理内存管理

4.2.1 物理页帧分配器

物理页帧分配器管理从内核镜像结束到 `MEMORY_END` 之间的可用物理页。初始化时，内核根据链接符号 `ekernel` 计算可分配起点，并与平台内存起点取较大值，避免把内核镜像、启动栈或已占用区域重新纳入页帧池。

当前实现采用栈式分配器：从连续页帧区间顺序分配新页，释放后的页号进入 `recycled` 栈，后续优先复用。这个策略实现简单，适合初赛阶段以正确性和可调试性为主的需求；页帧不足时返回 `None`，上层再转换为 `ENOMEM` 或相应错误路径。

4.2.2 物理页帧的生命周期管理

`frame_alloc` 返回 `FrameTracker`，构造时会清零整页，释放时通过 `Drop` 自动归还页帧。用户数据页和页表页都由 `FrameTracker` 持有，区别在于前者通常被 `MapArea.data_frames` 记录，后者由 `PageTable.frames` 记录。这样地址空间销毁、逻辑段拆分、`munmap` 或 `COW` 换页时，只要对应容器移除所有权，物理页就会自动进入回收路径。

三级页表本身也会消耗物理页：根页表至少占用一页，每遇到新的中间级索引还会再分配页表页。实际开销取决于虚拟地址分布，连续小进程通常只需要少量页表页，而 `mmap`、用户栈、动态链接器和高地址 `trampoline` 分散到不同区间时，会额外增加中间级页表页数量。

场景	持有者	释放条件
页表页	<code>PageTable.frames</code>	地址空间销毁或 LoongArch 页表延迟退休后释放。
普通用户页	<code>MapArea.data_frames</code>	逻辑段 <code>unmap</code> 、地址空间销毁或 COW 换页后引用归零。
共享映射页	<code>Arc<FrameTracker></code>	所有共享地址空间都解除映射后释放。

表 10 页帧生命周期入口

4.3 进程内存管理

4.3.1 虚拟内存区域

MapArea 是进程虚拟内存区域的基本单位。它不只记录虚拟页范围，还记录映射类型、权限、是否共享和已分配页帧集合。立即映射的区域通过 `push_empty_map_area` 建立全部页表项；懒分配区域通过 `push_map_area_lazy` 只记录范围，真正的物理页在首次访问缺页时分配。

为了支持 `munmap`、`mprotect` 和 `mmap` 覆盖语义，**MapArea** 可以按重叠区间拆分成左、中、右三段，并把对应的 `data_frames` 一起切分。这个设计避免了“一个 VMA 只能整体删除”的限制，也让部分解除映射后仍能保留剩余区间的权限和共享属性。

4.3.2 进程地址空间

创建新用户进程时，`from_elf_data` 解析 ELF 程序头，将每个 LOAD 段映射为用户 **MapArea**，再根据需要加载动态链接器并构造 `auxv`。用户栈、guard page、堆起点和 `mmap` 区间的相对位置已经在地址空间布局小节说明；本节关注的是这些区域如何由 **MemorySet** 记录并在 `execve`、`fork`、`brk`、`mmap` 路径中演进。

`fork` 路径通过 `from_existed_user` 创建子地址空间。只读页可以直接共享；可写页在父子两边都移除写权限并设置 COW 标志；尚未实际分配的懒分配页只复制虚拟范围，不复制物理页。这样普通 `fork` 不需要立即复制整个地址空间，只有父进程或子进程首次写共享页时才触发真正的数据复制。

4.4 缺页异常处理

4.4.1 进程缺页异常概述

用户态缺页由第三章的 `trap` 层识别后转交给 `MemorySet::handle_page_fault`。内存管理层先根据 `fault` 地址定位所属 **MapArea**，再检查该区域是否为用户映射、访问类型是否满足权限，最后根据页表项状态选择 COW 修复、懒分配或报错。缺页失败会返回错误，由 `trap` 层转化为 `SIGSEGV` 或任务退出。

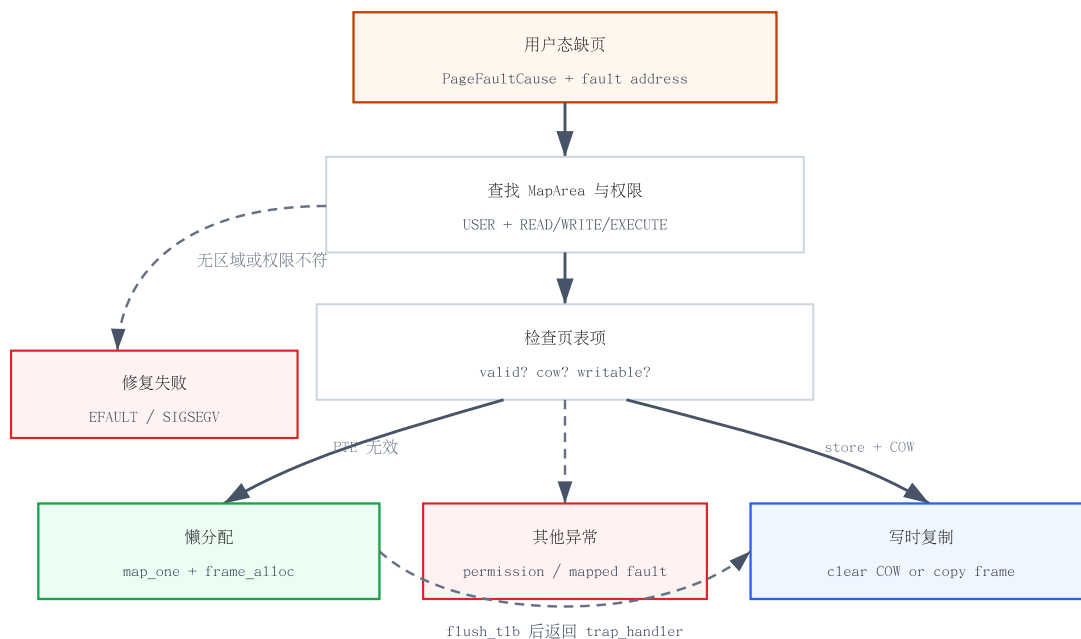


图 9 缺页异常处理流程

4.4.2 写时复制机制

写时复制只在写访问触发、页表项有效且带 COW 标志、所属 MapArea 允许写入时生效。若共享物理页的 Arc 引用计数为 1，说明已经没有其他地址空间共享该页，内核只需要恢复页表写权限并清除 COW 标志；若引用计数大于 1，则分配新页帧、复制旧页数据、替换当前地址空间的页表项和 data_frames 记录。

```

if is_store && pte.is_some_and(|p| p.is_valid() && p.is_cow()) {
    let old_frame = area.data_frames.get(&vpn).ok_or(Errno::EFAULT)?;
    if Arc::strong_count(old_frame) == 1 {
        page_table.modify_pte(vpn, PTEFlags::from(area_perm));
        page_table.clear_pte_cow(vpn);
    } else {
        area.remap_one_with_data(&mut page_table, vpn, old_frame.ppn().get_bytes_array()?);
    }
}

```

完成 COW 修复后必须刷新 TLB，否则硬件可能继续使用旧的只读页表项，导致同一地址反复触发写缺页。COW 的关键不是“延迟复制”本身，而是父子地址空间的页表权限、软件 COW 标志和物理页引用计数三者必须同步变化。

4.4.3 懒分配机制

懒分配用于匿名 mmap、堆扩展以及尚未触碰的用户区域。若 fault 地址位于合法 MapArea 内，但页表项不存在或无效，内核会根据 fault 类型推导所需权限：写访问需要 WRITE，极少情况下取指缺页需要 EXECUTE，读访问需要 READ。权限满足时，map_one 分配新物理页、写入页表项，并把页帧记录到 data_frames。

这种机制减少了大块匿名映射和堆扩展的初始成本，也避免 `fork` 时复制从未触碰过的页面。代价是用户指针访问不能简单地直接解引用用户地址，因为目标页可能尚未建立物理映射；这也是用户地址检查和 `copy` 接口需要主动触发缺页修复的原因。

4.5 内核动态内存分配

内核堆使用 `buddy_system_allocator::LockedHeap`，堆空间静态存放在 `.bss` 段中的 `HEAP_SPACE`，大小由 `KERNEL_HEAP_SIZE` 控制，目前为 64 MiB。初始化时，`init_heap` 将这段连续内存交给全局分配器，后续 `Arc`、`Vec`、`BTreeMap`、路径字符串、文件系统缓存元数据和任务结构都依赖该堆分配能力。

内核堆和物理页帧分配器职责不同。堆分配器面向内核对象和小粒度动态内存，页帧分配器面向页表页、用户数据页和大块页级映射。两者都可能因资源不足失败，但错误处理层次不同：堆分配失败会触发 `allocator panic`，页帧不足通常通过 `frame_alloc` 返回 `None` 并向系统调用路径报告 `ENOMEM`。

4.6 用户地址检查

用户态指针进入内核后，RespOS 不直接解引用用户虚拟地址，而是先通过 `check_user_readable` 或 `check_user_writable` 验证范围属于用户 `MapArea` 且权限满足要求，再调用 `ensure_user_page_access` 主动处理懒分配或 COW。实际拷贝时，内核逐页查页表，把用户虚拟地址翻译到物理页帧，再通过 `get_bytes_array` 完成跨页复制。

这种设计把“地址是否合法”和“页是否已经实际分配”分开处理。前者由 `MapArea` 权限判断保证，后者由缺页修复逻辑保证；如果任一阶段失败，系统调用返回 `EFAULT`。对于 `read`、`write`、`stat`、`poll`、`mmap` 等大量使用用户缓冲区的接口，这比在内核态触发 `page fault` 后再兜底更可控，也更容易定位 ABI 兼容问题。

本章小结

内存管理章的核心是把地址空间布局、页表元数据、物理页所有权和缺页修复分层。线性映射解决内核访问物理页的问题，页帧分配器决定页是否被占用，`MemorySet` 和 `MapArea` 则维护用户进程看到的虚拟内存语义。

第五章

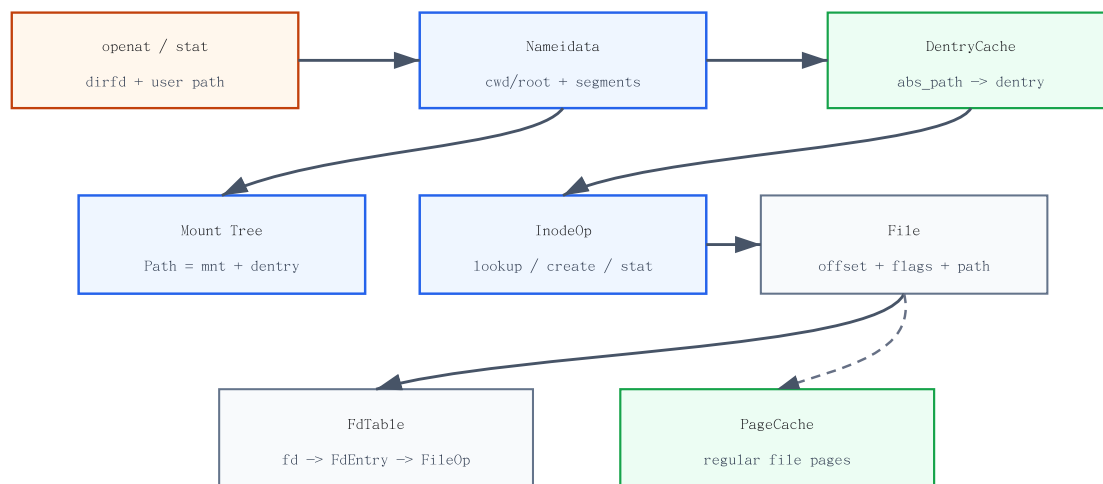
文件系统

文件系统模块把块设备、伪文件系统、设备文件和内核对象统一暴露为 Linux 风格路径与文件描述符。本章先说明 VFS 的抽象边界，再分别介绍 Ext4、procfs/devfs、页缓存和 fd 表如何协作支撑用户态程序。

5.1 虚拟文件系统

文件系统模块为用户态提供 Linux 风格路径、文件描述符、目录项和挂载语义。RespOS 没有把系统调用直接绑定到某一种磁盘格式，而是在 fs/vfs/ 中定义 `SuperBlockOp`、`InodeOp`、`Dentry` 和 `FileOp` 等抽象，再由 Ext4、procfs、devfs、pipe、stdio 和特殊 fd 对象分别实现对应能力。

VFS 的核心目标是把“路径解析”“文件对象语义”和“打开文件状态”分开。路径解析得到的是 `Path { mnt, dentry }`，表示某个挂载实例中的目录项；`InodeOp` 描述该对象支持的元数据和读写接口；`File` 保存一次打开后的偏移量、打开标志和页缓存引用；`FdTable` 再把用户可见的整数 fd 映射到 `FileOp` 对象。



路径解析负责“找到对象”，打开文件负责“保存一次打开状态”，fd 表负责“把整数句柄映射到 FileOp”。

图 10 VFS 路径解析与打开文件流程

5.1.1 SuperBlock

`SuperBlockOp` 表示一个文件系统实例，至少需要提供根 inode 和同步接口，并可选实现 `statfs`。Ext4、procfs 和 devfs 都通过不同的 super block 挂载到同一棵 mount tree 中，因此系统调用层可以统一调用 `Path.mnt.fs.statfs()` 获取文件系统统计信息，而不需要知道路径最终落在哪个后端。

根文件系统在 `init_root_fs` 中建立：内核先通过 `fs/ext4` 创建 Ext4 super block 和根 inode，把它作为 `/` 的 root mount 加入挂载树，然后再把 `procfs` 和 `devfs` 分别挂载到 `/proc` 与 `/dev`。

挂载树由 `VfsMount` 和 `Mount` 维护。`VfsMount` 保存一个文件系统实例的根目录项和 super block，`Mount` 记录它挂载在父文件系统的哪个 dentry 上。路径解析遇到挂载点时会切换到子挂载实例的根 dentry；处理 `..` 时如果已经位于当前挂载根，还需要回到父挂载点，避免路径穿越逻辑与普通目录父子关系混在一起。

5.1.2 Inode

`InodeOp` 表示文件系统对象本身，负责 `stat`、`read_at`、`write_at`、`truncate`、`lookup`、`readdir`、`create`、`link`、`unlink` 等语义。它不保存“这次打开文件的偏移量”，也不直接暴露 `fd`；这些状态属于 `File` 和 `FdTable`。这种拆分使同一个 inode 可以被多个文件描述符共享，同时每个打开文件仍有独立 offset 和 flags。

Ext4 inode 会将底层 `lwext4` 错误码映射为 Linux `errno`，并通过 inode cache 复用同一 inode 对象；常规文件 inode 还持有共享 `PageCache`。`procfs` 和 `devfs` 的 inode 则通常不对应磁盘块，而是根据当前任务、系统状态或设备对象动态生成目录项和文件内容。

5.1.3 Dentry

`Dentry` 是路径名缓存和目录树关系的承载对象，保存绝对路径、父目录、弱引用子目录项和可选 inode。路径解析时，`Nameidata` 从当前任务的 `cwd` 或 `root` 出发，按路径分量逐级查找；先查全局 dentry cache，未命中时调用当前目录 inode 的 `lookup`，再把新 dentry 插入父目录和缓存。

dentry 和 inode 的职责不同：dentry 关心“某个名字在目录树中的位置”，inode 关心“该对象支持哪些文件操作”。硬链接、挂载点和符号链接都会让名字关系变复杂，因此 VFS 需要 dentry 层来保存路径结构，而不能只靠 inode 号描述用户看到的文件树。

5.1.4 File

`File` 表示一次打开文件，内部保存 inode、当前 offset、打开标志、所属 `Path` 和可选页缓存。`read`、`write` 会根据 offset 调用页缓存或底层 inode，并在成功后推进 offset；`O_APPEND` 写入前会把 offset 调整到当前文件末尾；`O_TRUNC` 在打开时截断常规文件并同步调整页缓存长度。

`File` 实现了 `FileOp` trait，`fd` 表中存储的是 `Arc<dyn FileOp>`，通过 trait object 统一分派到常规文件、管道、设备等后端。这样 `read`、`write`、`poll`、`fcntl` 等系统调用只需要先通过 `fd` 找到 `FileOp`，再按对象能力分派，不需要在系统调用入口硬编码所有后端类型。

5.2 磁盘文件系统

5.2.1 Ext4 文件系统

RespOS 的磁盘文件系统后端基于 `lwext4_rust`。初始化时，`fs/ext4/mod.rs` 获取 `VirtIO block` 设备并交给 `Disk` 适配层；`Disk` 再把块设备的读、写和刷新能力转换成 `lwext4` 所需的 `KernelDevOp` 连续读写接口。

`Ext4SuperBlock::new` 创建 `Ext4BlockWrapper<Disk>` 时会完成 `lwext4` 挂载和 `superblock` 读取, 随后构造 `inode 2` 作为根 `inode`。`Ext4 inode` 通过 `lwext4 bindings` 执行目录查找、文件读写、创建、链接、重命名、符号链接和 `statfs` 等操作; 底层错误码由 `Ext4Inode::map_lwext4_err` 转成 `ENOENT`、`EIO`、`EEXIST`、`ENOTDIR`、`ENOSPC` 等内核 `Errno`, 未知错误统一按 `EIO` 处理。

为了减少重复构造 `inode` 对象, `Ext4` 层维护 `EXT4_INODE_CACHE`, 以 `inode` 号映射到弱引用。常规文件的 `Ext4Inode` 还关联一个共享 `PageCache`, 同一 `inode` 的多个 `File` 打开实例可以共享缓存页, 但每个 `File` 仍保留自己的 `offset` 和打开标志。对 `lwext4` 的修改操作通过 `EXT4_OP_LOCK` 串行化, 避免底层 `C` 库状态在并发路径中被破坏。

5.3 非磁盘文件系统

5.3.1 procfs

`procfs` 用于向用户态暴露内核和进程状态。初始化时, 内核在根文件系统中创建 `/proc` 挂载点, 再挂载 `ProcSuperBlock`; `/proc/self`、`/proc/self/fd`、`/proc/self/maps`、`/proc/self/smmaps`、`/proc/self/stat`、`/proc/cpuinfo`、`/proc/meminfo` 等文件由对应虚拟 `inode` 动态生成内容。

`procfs` 的文件多数没有持久化数据块。读取时 `inode` 根据当前任务、内存映射、`fd` 表或系统统计信息生成文本; 写入、链接和删除通常返回只读或不支持错误。这个后端主要服务 `libc`、`busybox` 和 `LTP` 对 Linux `/proc` 兼容性的依赖。

5.3.2 devfs

`devfs` 在 `/dev` 下提供设备和特殊文件入口, 包括 `/dev/null`、`/dev/zero`、`/dev/random`、`/dev/urandom`、`/dev/shm`、`/dev/misc/rtc`、`/dev/loop-control` 和 `loop0` 等。它同样通过 `VFS` 挂载树接入, 但具体 `inode` 的读写语义由设备类型决定, 例如 `/dev/null` 读取返回 `EOF`、写入丢弃数据, `/dev/zero` 读取填充零字节。

`devfs` 的意义不只是提供几个路径名, 而是把设备对象纳入统一的 `fd` 和路径模型。用户程序可以用普通 `openat`、`read`、`write`、`stat` 访问设备文件, 系统调用层无需为每个设备额外设计入口。

5.4 页缓存

页缓存位于 `VFS` 与磁盘后端之间, 按 `inode` 共享常规文件的数据页。`PageCache` 以页号索引缓存页, 每页包含 4 KiB 数据、`dirty` 标志、`LRU generation` 和队列状态。读取未命中时, 页缓存会在锁外调用底层 `inode` 的 `read_at` 加载数据; 写入时先修改缓存页并标记 `dirty`, `fsync` 或 `File drop` 时再通过 `sync` 写回底层文件。

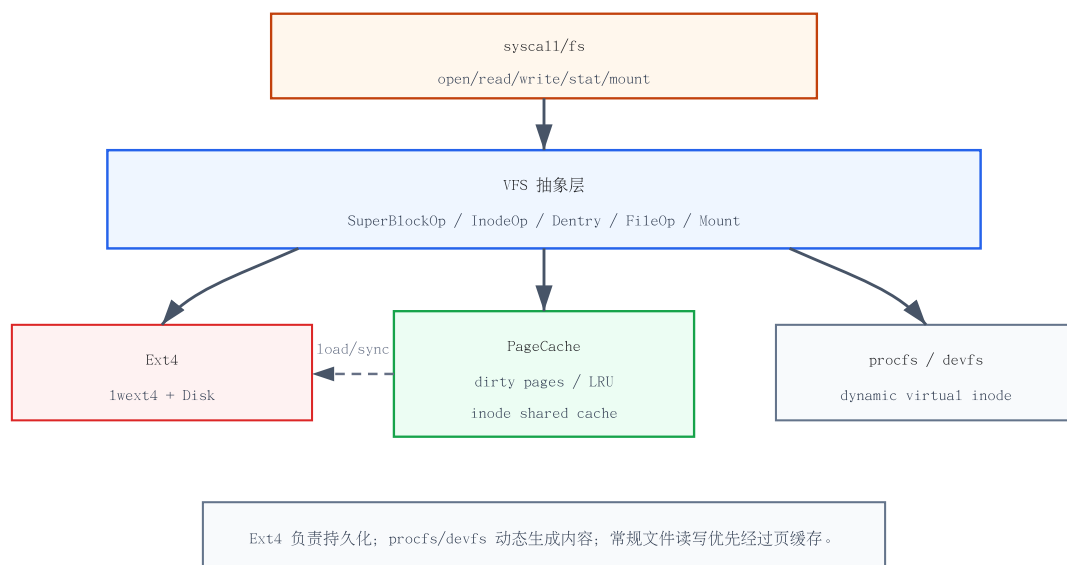


图 11 文件系统后端与页缓存关系

全局页缓存通过 `PAGE_CACHE_LRU` 和 `PAGE_CACHE_PAGE_COUNT` 控制规模。只有未 `dirty`、`generation` 未变化且没有额外强引用的页才能被回收；`dirty` 页需要先写回，仍被文件或其他路径持有的页也不能直接丢弃。这个策略避免了简单粗暴释放缓存导致的数据丢失，同时让常规文件重复读写可以绕开底层磁盘 I/O。

5.5 文件描述符表

`FdTable` 是进程资源的一部分，保存 `Vec<Option<FdEntry>>`、下一个可用 `fd` 和 `RLIMIT_NOFILE` 边界；它也对第二章 TCB 中的进程资源字段。新任务创建时默认拥有 0、1、2 三个标准 `fd`；`openat` 分配最小可用 `fd`，`dup/fcntl` 可以从指定下界查找空位，`close` 会释放表项并回退 `next_fd`。`execve` 前会根据 `O_CLOEXEC` 清理需要关闭的描述符。

`FdEntry` 保存 `Arc<dyn FileOp>` 和 `fd flags`。`fork` 时 `fd` 表会复制表项，但底层 `FileOp` 通过 `Arc` 共享，因此父子进程可以共享同一个打开文件对象和 `offset`；这符合大量 Unix 程序对 `fork` 后文件偏移共享的预期。对于 `pipe`、`socket`、`eventfd`、`timerfd` 等特殊对象，`fd` 表不需要知道内部结构，只负责持有和查找 `FileOp`。

本章小结

文件系统章的核心设计是把路径、挂载、`inode`、打开文件和 `fd` 表拆开：VFS 保持统一语义，Ext4 负责持久化，`procfs/devfs` 负责兼容性入口，页缓存承担常规文件的性能边界。这样的分层让后续增加 `pipe`、`socket`、`eventfd` 等对象时可以复用 `fd` 模型，而不需要反复修改系统调用入口。

第六章

进程间通信

RespOS 的 IPC 支持围绕 Linux 用户程序最常依赖的控制流通知、字节流通信、共享内存和用户态同步展开。本章把信号、管道、System V 共享内存和 `futex` 放在同一章中说明，重点关注它们如何与任务状态、文件描述符表和地址空间管理交叉协作。

6.1 信号机制

信号机制用于把异步事件递送给用户任务，是进程控制、异常通知、定时器和阻塞系统调用中断的共同基础。RespOS 将信号状态拆成三部分：`SigPending` 保存未决信号位图、当前屏蔽集和 `SigInfo`；`SigHandler` 保存每个信号的处理动作；任务控制块负责在阻塞、停止、继续和退出路径中与调度器协作。

信号不会在发送时立即改写用户上下文。发送路径只负责构造 `SigInfo` 并放入目标任务的 pending 集合；真正递送发生在第三章所述的 `trap` 返回用户态之前，由 `handle_signal` 检查未被屏蔽的未决信号。这样可以保证内核总是在一个明确的边界修改 `TrapContext`，避免在任意内核执行点强行插入用户 `handler`。

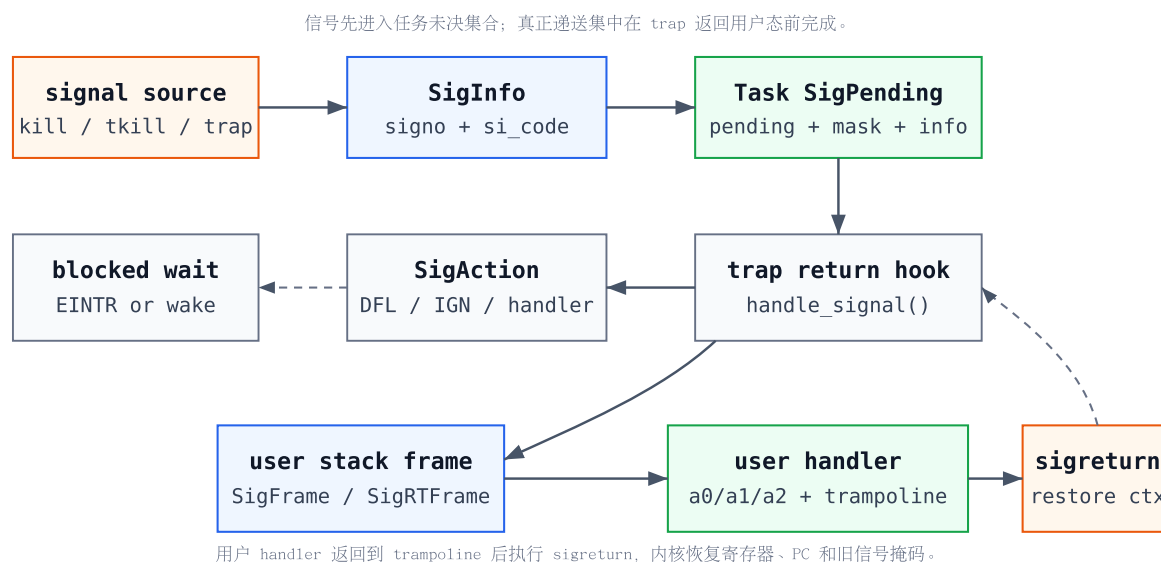


图 12 信号递送与返回流程

6.2 信号传输

信号来源主要包括三类。第一类是用户显式发送，`kill` 按 PID、进程组或全体任务选择目标，`tkill` 按 TID 发送，`tgkill` 同时校验 TGID 与 TID，避免把信号误送给已经复用的线程号。第二类是内核异常生成，例如非法指令导致任务终止、缺页失败递送 `SIGSEGV`、管道破裂递送 `SIGPIPE`。第三类来自时间和等待路径，例如 `real timer` 到期、阻塞系统调用被信号打断。

传输阶段还需要处理 Linux 权限和线程语义。`kill` 会检查发送者和目标任务的 `uid`、`euid`、`suid`，非特权任务不能随意向其他用户进程发送信号；`SIGKILL` 和 `SIGSTOP` 不能被屏蔽；`SIGCONT` 到达停止任务时会更新 `wait` 事件并唤醒任务。面向线程的信号则进入具体 TID 的 `pending` 集合，面向进程的信号可以根据线程组选择可接收的线程。

接口	职责
<code>kill</code>	按 PID、进程组或全体任务选择目标，并执行权限检查。
<code>tkill</code> / <code>tgkill</code>	向具体线程发送信号，其中 <code>tgkill</code> 额外校验线程组。
<code>rt_sigaction</code>	安装或查询用户 handler，禁止修改 <code>SIGKILL</code> 与 <code>SIGSTOP</code> 。
<code>rt_sigprocmask</code>	修改当前任务屏蔽集，并自动移除不可屏蔽信号。
<code>rt_sigsuspend</code>	临时替换屏蔽集并进入可中断等待，醒来后返回 <code>EINTR</code> 。
<code>rt_sigtimedwait</code>	从指定集合中消费 <code>pending</code> 信号，可带超时。
<code>sigaltstack</code>	设置备用信号栈，供 <code>SA_ONSTACK</code> handler 使用。
<code>sigreturn</code>	从用户栈读取信号帧，恢复寄存器、PC 和旧屏蔽集。

表 11 信号相关系统调用职责

6.3 信号处理

信号处理阶段由 `SigAction` 决定最终动作。若 handler 为 `SIG_IGN`，内核直接忽略；若为 `SIG_DFL`，根据默认动作执行终止、停止、继续或忽略；若为用户函数地址，内核会在用户栈或备用信号栈上构造信号帧，并把 `TrapContext` 的返回地址改写到用户 handler。

RespOS 支持普通信号帧和 `SA_SIGINFO` 实时信号帧。普通帧保存 `SigContext`，记录被打断时的通用寄存器、返回 PC 和旧屏蔽集；`SA_SIGINFO` 路径额外压入 `LinuxSigInfo` 和 `UContext`，并把 handler 参数设置为 `signo`、`siginfo_t*`、`ucontext_t*`。栈帧开头带 `FrameFlags`，`sigreturn` 依靠它区分普通帧和 RT 帧。

标志	效果
<code>SA_SIGINFO</code>	使用 RT 信号帧，向 handler 传入 <code>siginfo_t*</code> 和 <code>ucontext_t*</code> 。
<code>SA_NODEFER</code>	handler 执行时不自动屏蔽当前信号。
<code>SA_RESETHAND</code>	信号递送后把该信号动作恢复为默认值。
<code>SA_ONSTACK</code>	在备用信号栈上执行 handler。

表 12 信号处理相关 SA 标志

用户 handler 返回到架构相关的 `trampoline` 后进入 `sys_sigreturn`，内核再从用户栈读回保存的上下文，恢复 `TrapContext` 和信号屏蔽集。

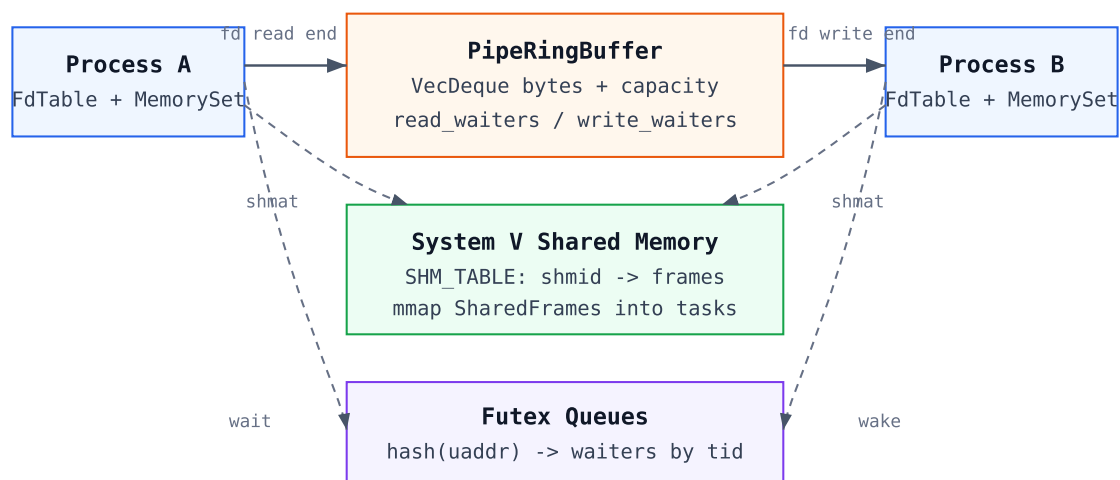
6.4 管道 (Pipe)

6.4.1 管道设计

管道是通过文件描述符暴露的字节流 IPC。pipe2 创建一个共享 PipeRingBuffer，并返回读端和写端两个 Pipe 对象；二者都实现 FileOp，复用第五章介绍的 trait object 分派模型，因此可以被 read、write、poll、fcntl、close 和 splice 等文件系统路径统一处理。管道不可 seek，相关接口返回 ESPIPE。

PipeRingBuffer 使用 VecDeque<u8> 保存数据，并维护容量、读端关闭标志、写端关闭标志、读等待队列和写等待队列。默认容量受 PIPE_BUFFER_SIZE 和 /proc/sys/fs/pipe-max-size 影响，F_GETPIPE_SZ / F_SETPIPE_SZ 可以查询或调整容量；调小容量时若小于当前已缓存字节数，会返回 EBUSY。

```
struct PipeRingBuffer {
    buffer: VecDeque<u8>,
    capacity: usize,
    read_closed: bool,
    write_closed: bool,
    read_waiters: VecDeque<usize>,
    write_waiters: VecDeque<usize>,
}
```



管道通过 fd 传递字节流；共享内存通过同一批页帧共享数据；futex 只在用户地址上建立等待/唤醒关系。

图 13 IPC 对象与任务关系

6.4.2 读端与写端通信

读管道时，如果缓冲区已有数据，内核直接拷贝并唤醒一个写等待者；如果缓冲区为空且写端已经关闭，则返回 0 表示 EOF；如果缓冲区为空但写端仍存在，当前任务进入可中断阻塞状态，并把 TID 放入 read_waiters。写管道时，如果缓冲区有空间，内核写入数据并唤醒一个读等待者；如果缓冲区满，写端进入 write_waiters；如果读端已经关闭，则返回 EPIPE。

管道阻塞路径必须和信号机制配合。任务进入等待队列前会设置 interruptible 标志，醒来后检查 is_interrupted 和 check_signal_interrupt，若被信号打断则返回 EINTR 并从等待队列移除自身。这样 read、write、poll、splice 等接口既能等待对端，又不会吞掉用户期望观察到的信号。

6.5 System V IPC 机制

System V IPC 当前主要覆盖共享内存子集，目标是满足 libc、busybox 和 LTP 中常见的 `shmget`、`shmat`、`shmctl`、`shmdt` 路径。它与管道的区别在于：管道传递的是内核缓冲区中的字节流，共享内存则把同一批物理页映射到多个进程地址空间中，读写动作由用户态直接完成。

6.5.1 System V IPC 对象

全局 `SHM_TABLE` 保存所有共享内存段，表项由 `shmid` 索引。`ShmSegment` 是表中的共享内存对象，保存段大小和一组 `Arc<FrameTracker>`；`shmget` 会按页对齐请求长度，分配对应数量的物理页帧，并把这些页帧放入新的 `ShmSegment`。这些页帧不属于某个单独进程，而是由共享内存对象持有，进程只是在自己的地址空间中引用它们。

`shmat` 从 `SHM_TABLE` 查出段对象后，把其中的页帧作为 `MmapBacking::SharedFrames` 映射进当前任务的 `MemorySet`。未设置 `SHM_RDONLY` 时映射带 `READ | WRITE | USER` 权限，设置只读标志时去掉写权限。映射完成后刷新 TLB，使用户态立即看到新的地址空间状态。

6.5.2 IPC key 管理

目前实现对 key 语义采用保守子集：`IPC_PRIVATE` 总是创建新段；非私有 key 若没有 `IPC_CREAT` 则返回 `ENOENT`。全局表用递增的 `next_id` 分配 `shmid`，而不是维护完整 key 到对象的复用关系。这种做法可以覆盖初赛阶段大量“创建、附着、读写、删除”的共享内存用例，但还不是完整 System V IPC 命名空间实现。

完整 Linux 语义还需要处理 key 复用、权限位、`IPC_EXCL`、引用计数、namespace 和 `shmid_ds` 元数据。RespOS 当前把这些复杂度暂时压缩在 `syscall/ipc.rs` 中，后续若需要补齐 System V semaphore 或 message queue，可以继续沿用“全局表 + ID 分配 + 对象元数据”的组织方式。

6.5.3 System V 共享内存

共享内存的生命周期由四个系统调用串起来。`shmget` 创建段并返回 `shmid`；`shmat` 把段映射到当前进程，地址可以由内核选择，也可以由用户指定页对齐地址；`shmdt` 按起始地址移除对应 VMA 并刷新 TLB；`shmctl(shmid, IPC_RMID, NULL)` 用 `IPC_RMID` 命令从全局表删除段。由于实际数据页由 `Arc<FrameTracker>` 持有，已经映射到进程地址空间中的共享页会在引用归零后释放。

这种设计和普通匿名 `mmap` 的关键差异在于页帧来源。匿名映射通常在缺页时给每个进程分配自己的页；System V 共享内存则在段创建时分配一组共享页，多个进程的页表项指向同一批物理页。因此一个进程写入共享页后，其他已附着进程可以直接通过自己的虚拟地址观察到变化。

6.6 Futex 同步

futex 不是传统 System V IPC，但它是用户态线程库实现 `mutex`、`condvar` 和 `join` 语义的重要基础。RespOS 的 `do_futex` 在任务模块中解析 `FUTEX_WAIT`、`FUTEX_WAKE`、`FUTEX_REQUEUE`、`FUTEX_CMP_REQUEUE`、`FUTEX_WAIT_BITSET` 和 `FUTEX_WAKE_BITSET`，系统调用层只负责传入原始参数。

futex 等待队列按用户地址哈希到 256 个桶。私有 futex 的 key 包含线程组号和用户地址，共享 futex 当前使用全局 scope；等待前内核会先读取用户地址的实际值，若和期望值不一致则

返回 `EAGAIN`，避免错过用户态已经完成的状态变化。等待任务同样以可中断方式阻塞，信号到达时返回 `EINTR`。

本章小结

进程间通信章围绕三类共享边界展开：信号共享的是控制流事件，管道共享的是内核字节缓冲，共享内存共享的是物理页帧，`futex` 则在用户地址上建立等待/唤醒关系。这些机制都必须和任务状态、信号中断、文件描述符表或地址空间管理协作，才能呈现接近 Linux 的用户态行为。

第七章

时钟模块

时钟模块为内核提供三类能力：读取当前时间、安排下一次时钟中断、向用户态暴露 Linux 时间相关系统调用。RespOS 当前没有单独实现复杂的时间轮或红黑树定时器队列，而是以硬件周期中断作为驱动，在 trap 返回、调度和阻塞等待边界检查任务定时器与 POSIX timer。这样的设计实现简单，足以覆盖初赛阶段 libc、busybox、LTP 对 `clock_gettime`、`nanosleep`、`setitimer`、`timer_create` 和 `timerfd` 的主要依赖。

从模块边界看，架构层负责读取硬件计数器并设置下一次中断；系统调用层负责 Linux ABI 参数转换、权限检查和错误码；任务模块保存 per-task interval timer；文件系统 fd 模型承载 `timerfd`。第七章沿这条路径展开，说明 RespOS 如何把底层 tick 转换为用户可见时间，并把时间到期事件递送为信号、可读 fd 或调度唤醒。

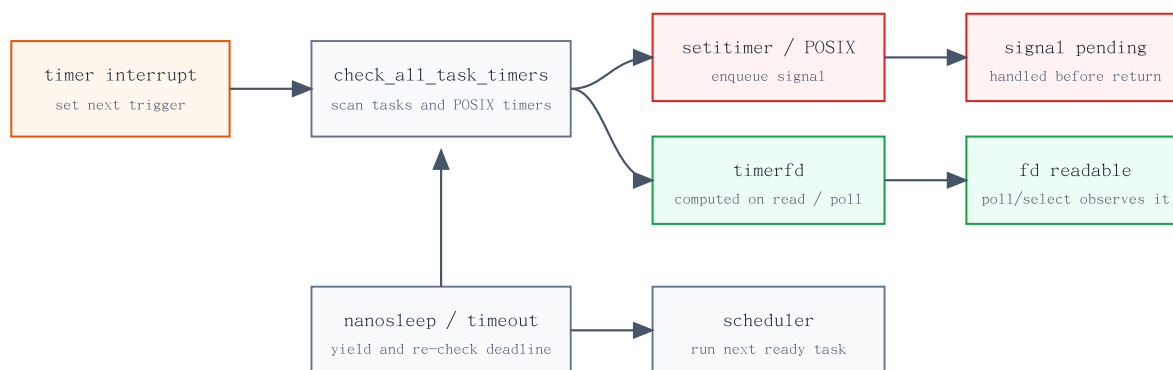


图 14 定时事件处理路径

7.1 时钟源与时间尺度

RespOS 在 RISC-V 64 与 LoongArch 64 上分别读取平台计数器。RISC-V 路径通过 `time::read()` 获取 `mtime`，并通过 SBI `set_timer` 设置下一次 supervisor timer；LoongArch 路径通过 `rdtime.d` 读取稳定计数器，再使用平台 timer 接口设置 one-shot 中断。两套架构向上暴露相同的 `get_time_ms`、`get_time_us`、`get_timeout_ms` 和 `set_next_ti_trigger` 接口，因此 trap、系统调用和任务模块不需要关心底层计数器差异。

当前实现把时钟频率分成三类，避免同一个频率常量同时承担所有语义。`HARDWARE_CLOCK_FREQ` 用于 timer interrupt 编程和 timeout/deadline 判断，保证阻塞等待与调度节奏跟真实硬件计数器一致；`USER_CLOCK_FREQ` 用于 `gettimeofday`、`clock_gettime` 等用户可见时间；`ACCOUNTING_CLOCK_FREQ` 用于 `times`、`getrusage` 等 CPU 时间记账近似。LoongArch 启动时还会读取 `CPUCFG` 作为诊断输出，但运行时仍以 board 配置中的频率策略为准。

时间尺度	接口	用途
硬件时间	<code>get_timeout_ms/us</code>	<code>timer interrupt</code> 、 <code>nanosleep</code> 、 <code>futex</code> 超时、 <code>socket</code> 超时和 <code>interval timer deadline</code> 。
用户时间	<code>get_time_ms/us</code>	<code>gettimeofday</code> 、 <code>clock_gettime</code> 、文件时间戳和用户可见 <code>wall/monotonic</code> 时间。
记账时间	<code>get_accounting_ms/us</code>	<code>times</code> 、 <code>getrusage</code> 相关 CPU tick 统计，目前仍是调度记账的近似实现。

表 13 时钟尺度与主要用途

7.2 周期中断与调度协作

架构初始化阶段会打开 `timer interrupt`，并在进入任务调度前调用 `set_next_ti_trigger`。每次用户态 `timer interrupt` 到来后，`trap` 处理流程先设置下一次触发点，再调用 `check_all_task_timers` 检查所有任务的 `interval timer` 和 `POSIX timer`，最后让当前任务主动让出 CPU。这样时钟中断同时承担两个职责：提供抢占式调度的基本节拍，以及推进用户态可观察的定时事件。

`check_all_task_timers` 会遍历 `TASK_MANAGER` 中仍可访问的任务对象。对每个任务，任务模块检查 `ITIMER_REAL`、`ITIMER_VIRTUAL` 和 `ITIMER_PROF` 三个 `interval timer`；系统调用时间模块检查属于该线程组的 `POSIX timer`。若定时器已经到期，内核并不直接跳转到用户 `handler`，而是构造 `SigInfo` 放入任务 `pending` 集合，后续仍由第六章描述的信号递送路径在 `trap` 返回用户态前统一修改 `TrapContext`。

代码上，这个路径非常集中：`trap` 层只负责重新设置硬件触发点并让出 CPU，实际定时器扫描收敛到 `check_all_task_timers`。

```
Trap::Interrupt(Interrupt::SupervisorTimer) => {
    set_next_ti_trigger();
    check_all_task_timers();
    yield_current_task();
}

pub fn check_all_task_timers() {
    TASK_MANAGER.for_each(|task| {
        task.check_real_timer();
        check_posix_timers(task);
    });
}
```

对象	状态位置	到期动作
setitimer	TaskControlBlock.itimers	递送 SIGALRM、SIGVTALRM 或 SIGPROF，周期定时器重新设置 deadline。
timer_create	全局 POSIX_TIMERS 表	按 sigevent 中的信号号递送信号，周期定时器更新下一次到期时间。
timerfd	TimerFdState	不主动递送信号；read 或 poll 根据当前时间计算 pending expirations。
nanosleep	当前任务阻塞循环	到期后返回 0；被信号打断时写回剩余时间并返回 EINTR。

表 14 定时事件到期后的处理方式

7.3 用户可见时间接口

时间相关系统调用主要位于 `os/src/syscall/time.rs`。 `gettimeofday` 返回 `realtime_us` 转换得到的 `TimeVal`，并兼容历史遗留的 `timezone` 参数；系统时区固定为 UTC。 `clock_gettime` 支持 `CLOCK_REALTIME`、`CLOCK_MONOTONIC`、`CLOCK_BOOTTIME`、`CLOCK_TAI`、`coarse` 时钟以及进程/线程 CPU time id 等常见 clock id，其中 `realtime` 系列会叠加 `REALTIME_OFFSET_US`，`monotonic/boottime/CPU time` 当前使用用户时间近似。

`clock_settime` 只允许 `root` 权限调整 `CLOCK_REALTIME` 或 `CLOCK_REALTIME_ALARM`，实现方式不是修改硬件计数器，而是更新 `realtime offset`；非特权任务会得到 `EPERM`。 `adjtimex` 与 `clock_adjtime` 保存一份 `Timex` 状态，并在查询时刷新当前时间，用于满足 `libc` 和 `LTP` 对时间校准接口的结构体读写要求。

接口	当前语义
<code>gettimeofday</code>	返回 <code>realtime</code> 时间； <code>timezone</code> 参数若非空则返回 UTC 配置。
<code>clock_gettime</code>	支持 <code>realtime</code> 、 <code>monotonic</code> 、 <code>boottime</code> 、 <code>coarse</code> 、 <code>TAI</code> 和 <code>CPU time</code> 相关 clock id。
<code>clock_getres</code>	对支持的 clock id 返回 1 ns 名义分辨率。
<code>clock_settime</code>	仅 <code>root</code> 可调整 <code>realtime offset</code> ，不修改底层硬件计数器。
<code>adjtimex</code> / <code>clock_adjtime</code>	维护 <code>Timex</code> 状态并进行基本权限与参数校验。
<code>times</code>	返回当前任务 elapsed tick 和已回收子任务 tick，目前 CPU 时间仍为近似记账。

表 15 主要时间系统调用

7.4 睡眠与超时

`nanosleep` 将用户传入的 `TimeSpec` 校验并转换成毫秒 `deadline`，然后在循环中比较 `get_timeout_ms` 与起始时间。等待期间当前任务被标记为 `interruptible`，并主动 `yield_current_task` 让出 CPU；如果睡眠过程中收到信号，内核清理 `interrupted` 状态，按

Linux 语义把剩余时间写回 `rem`，并返回 `EINTR`。`clock_nanosleep` 当前覆盖 `CLOCK_REALTIME` 与 `CLOCK_MONOTONIC` 的相对睡眠语义，绝对时间睡眠暂时返回 `EINVAL`。

同一套 `timeout` 时间尺度也被复用到 `futex`、`poll/pselect`、`socket` 收发超时等路径。这样做的好处是所有阻塞接口都基于硬件时间判断 `deadline`，不受用户可见 `realtime offset` 调整影响；代价是当前等待实现多以“可中断等待 + 让出 CPU + 再检查时间”的方式推进，还没有集中式睡眠队列按最近 `deadline` 精确唤醒。

7.5 Interval Timer 与 POSIX Timer

`setitimer/getitimer` 使用 `TaskControlBlock` 内部的 `TaskTimers` 保存三个 `interval timer`。每个 `timer` 只需要两个原子字段：绝对到期时间 `deadline_ms` 和周期 `interval_ms`。当 `check_itimer` 发现当前硬件 `timeout` 时间已经越过 `deadline` 时，会用 `compare-exchange` 把一次性 `timer` 清零，或把周期 `timer` 推进到下一次 `deadline`，然后向任务递送对应信号。

```
struct IntervalTimer {
    deadline_ms: AtomicUsize,
    interval_ms: AtomicUsize,
}

fn fields(&self, which: usize) -> Option<(&AtomicUsize, &AtomicUsize, Sig)> {
    match which {
        0 => Some((&self.timers[0].deadline_ms, &self.timers[0].interval_ms, Sig::SIGALRM)),
        1 => Some((&self.timers[1].deadline_ms, &self.timers[1].interval_ms, Sig::SIGVTALRM)),
        2 => Some((&self.timers[2].deadline_ms, &self.timers[2].interval_ms, Sig::SIGPROF)),
        _ => None,
    }
}
```

POSIX timer 由全局 `POSIX_TIMERS` 表管理，`timer id` 递增分配，表项记录 `owner TGID`、`clock id`、递送信号号、`deadline` 和 `interval`。`timer_create` 当前支持 `SIGEV_SIGNAL` 子集，默认信号为 `SIGALRM`；`timer_settime` 支持相对时间和 `TIMER_ABSTIME`，`timer_gettime` 返回剩余时间与周期，`timer_delete` 按 `owner TGID` 校验后删除表项。到期检查时，内核只处理属于当前任务线程组的 `timer`，避免一个任务错误消费其他进程的定时事件。

7.6 timerfd

`timerfd` 把定时器包装成文件描述符，复用第五章的 `fd` 表和 `FileOp trait object` 模型。`timerfd_create` 校验 `clock id` 与 `O_NONBLOCK/O_CLOEXEC` 标志后创建 `TimerFd`；`timerfd_settime` 把用户传入的 `ITimerSpec` 转换为 `deadline` 与 `interval`；`timerfd_gettime` 返回当前剩余时间。

与 `signal timer` 不同，`timerfd` 不在中断路径主动入队事件，而是在 `read`、`poll` 或 `read_ready` 时根据当前时间即时计算已经到期但尚未消费的次数。`read` 要求用户缓冲区至少容纳一个 `u64`，成功后写回 `pending expirations` 并把 `consumed` 计数推进到最新状态；若尚未到期，则任务进入可中断等待并让出 CPU，收到信号时返回 `EINTR`。这种实现让 `timerfd` 可以自然接入 `poll/select` 等 `fd` 复用路径，同时避免为每个 `timerfd` 维护独立内核事件队列。

时钟模块的核心是把底层硬件计数器抽象成统一的时间读取、timeout 判断和 timer interrupt 编程接口。当前实现用周期中断驱动调度和定时器检查，时间系统调用负责 Linux ABI 兼容，interval timer/POSIX timer 通过信号递送，timerfd 则通过 fd 可读状态暴露到用户态。

第八章

网络模块

网络模块为用户态提供 Linux 风格 socket 接口，使用 `smoltcp` 承载 TCP/UDP 协议状态机。初赛阶段暂不接入真实网卡，优先覆盖 `libc`、`busybox` 和 `LTP` 中常见的本地网络行为：创建 socket、绑定地址、回环连接、收发数据、`poll` 可读写状态、设置常见 socket option。当前主要支持 IPv4 回环通信（127.0.0.1）、TCP 流式套接字、UDP 数据报套接字，以及轻量的 `AF_UNIX` socketpair。

整体调用链从系统调用层进入 `syscall/net.rs`，完成用户指针拷贝、`sockaddr_in` 转换、fd 校验和错误码映射；随后进入 `net/socket.rs` 的 `Socket` 对象。`Socket` 实现 `FileOp`，因此可以像普通文件一样放入 fd 表，并被 `read`、`write`、`poll`、`close` 等通用路径使用。真正的 TCP/UDP 协议操作由 `net/tcp.rs` 和 `net/udp.rs` 封装到全局 `smoltcp SocketSet` 中，再通过回环设备推进收发。

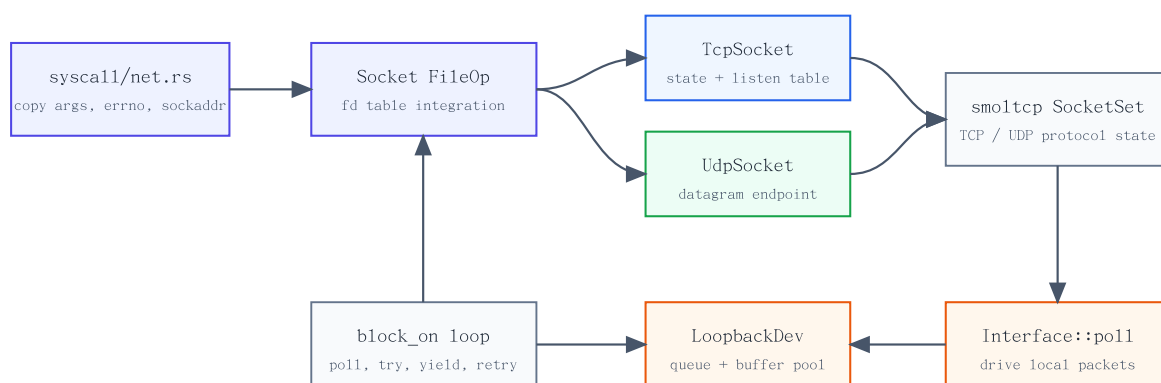


图 15 网络栈调用路径

8.1 Socket 套接字

`socket()` 系统调用首先解析 `domain`、`type` 和 `protocol`。RespOS 当前接受 `AF_INET + SOCK_STREAM + TCP`、`AF_INET + SOCK_DGRAM + UDP` 以及 `AF_UNIX + SOCK_STREAM/SOCK_DGRAM` 的 socketpair 子集；`AF_INET6` 返回 `EAFNOSUPPORT`，`SOCK_RAW` 返回 `EPROTONOSUPPORT`。`SOCK_NONBLOCK` 和 `SOCK_CLOEXEC` 会在创建时写入 `Socket` 标志，并同步影响后续阻塞等待和 `execve` 时的 fd 清理。

`Socket` 内部通过 `SocketInner` 分派到 `TcpSocket`、`UdpSocket` 或 `UnixSocket`。TCP 和 UDP 对象把协议状态保存在 `smoltcp socket handle`、地址字段和若干原子标志中；`UnixSocket` 只用于 socketpair，用两个共享的 `VecDeque<u8>` 实现本地字节队列。由于 `Socket` 实现了 `FileOp`，普通 `read/write` 会分别映射到 TCP 字节流、已连接 UDP 数据报或 UNIX pair 的收发；`sendto/recvfrom`、`sendmsg/recvmsg` 则在系统调用层额外处理地址和 `iovec`。

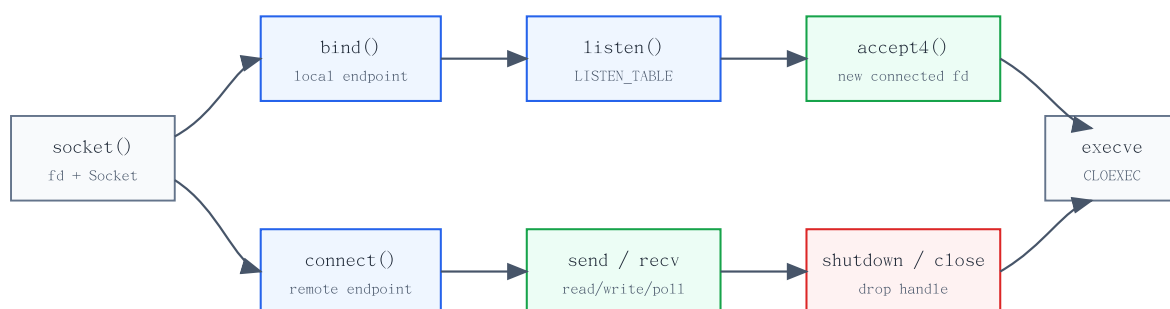


图 16 socket 从创建到收发的生命周期

Socket 的分派逻辑直接体现在构造函数中：协议选择失败时尽早返回 Linux 风格错误码，成功后统一接入 fd 表。

```

let inner = match (&domain, socket_type) {
  (SocketDomain::AF_UNIX, SocketKind::SOCK_STREAM | SocketKind::SOCK_DGRAM) => {
    SocketInner::Unix(UnixSocket::new())
  }
  (SocketDomain::AF_INET, SocketKind::SOCK_STREAM) =>
    SocketInner::Tcp(TcpSocket::new()),
  (SocketDomain::AF_INET, SocketKind::SOCK_DGRAM) => SocketInner::Udp(UdpSocket::new()),
  (SocketDomain::AF_INET6, _) => return Err(Errno::EAFNOSUPPORT),
  (_, SocketKind::SOCK_RAW) => return Err(Errno::EPROTONOSUPPORT),
};

```

接口	当前实现	说明
socket	AF_INET TCP/UDP, 部分 AF_UNIX	创建 Socket 并放入 fd 表, 支持 SOCK_NONBLOCK 和 SOCK_CLOEXEC。
socketpair	AF_UNIX stream/dgram	创建两个内核内存队列连接的 socket, 主要服务本地 IPC 兼容性。
bind / connect	IPv4 sockaddr_in	未指定地址会在协议层落到回环地址; UDP connect 记录默认对端。
listen / accept4	TCP	监听端口注册到 LISTEN_TABLE, accept 返回新的已连接 socket fd。
sendto / recvfrom	TCP/UDP/UNIX	系统调用层负责用户缓冲区拷贝和地址写回。
sendmsg / recvmsg	基础 iovec	支持分散/聚集数据拷贝, 不支持 OOB 和 error queue。
setsockopt / getsockopt	常见 SOL_SOCKET/TCP/IP 选项	覆盖 SO_REUSEADDR、buf size、timeout、TCP_NODELAY、IP_TTL 等。

表 16 socket 系统调用支持情况

8.2 回环网络设备

RespOS 当前网络设备是 `LoopbackDev`，实现 `smoltcp::phy::Device trait`。设备使用 `Medium::Ip`，接口地址配置为 `127.0.0.1/8`，所有发出的 IP 包不会进入真实网卡，而是被写入设备内部队列；下一次 `Interface::poll()` 时，这些包再作为接收包返回给 `smoltcp`。这种设计能在没有 `VirtIO-net` 驱动的情况下验证 `socket ABI` 和 `TCP/UDP 状态机`，也适合初赛阶段大量本机回环测例。

回环设备内部维护两个结构：`queue` 保存已经发送、等待接收的数据包；`pool` 保存可复用缓冲区，减少频繁分配。发送路径通过 `TxToken::consume` 取得缓冲区并让 `smoltcp` 填充包内容，随后补齐 IPv4/TCP 校验和并推入接收队列。接收路径通过代码中当前命名的 `RxTokenScoop` 把队列中的包交给 `smoltcp`，令牌析构时再把缓冲区回收到 `pool`。

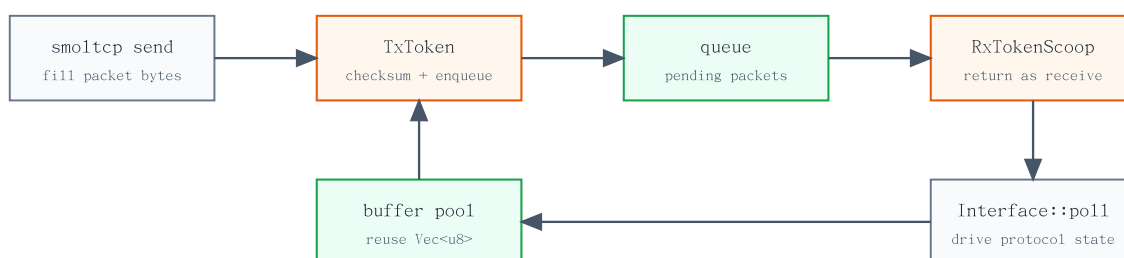


图 17 LoopbackDev 收发闭环

网络栈初始化时会先触发两个全局管理对象：`SOCKET_SET_INNER` 保存所有 `smoltcp socket`，`LISTEN_TABLE` 保存 `TCP` 监听端口。随后初始化回环设备和回环接口，即 `LOOPBACK_DEV` 与 `LOOPBACK_IFACE`。之后 `TCP/UDP` 的阻塞等待、`poll` 状态查询和 `connect/listen` 推进都会反复调用 `poll_interfaces`，由它带着当前时间戳执行 `Interface::poll()`。换言之，网络协议栈不是依靠独立网络中断推进，而是在系统调用和等待循环中主动轮询。

```

pub fn init() {
    let _ = &*SOCKET_SET_INNER;
    let _ = &*LISTEN_TABLE;
    let _ = &*LOOPBACK_DEV;
    let _ = &*LOOPBACK_IFACE;
}

pub fn poll_interfaces() {
    SOCKET_SET_INNER.lock().poll_interfaces();
}

```

8.3 传输层：UDP 与 TCP

UDP 是无连接数据报协议，对应 `UdpSocket`。构造 `UDP socket` 时，内核同步创建 `smoltcp::socket::udp::Socket` 并注册到全局 `SocketSet`。`bind` 会记录本地地址和端口，若用户传入端口 0 则分配临时端口；若地址为 `0.0.0.0`，协议层会使用回环地址作为实际绑定地址。`connect` 不建立握手，只保存默认远程端点，使后续 `write/read` 可以复用类似流式接口；未 `connect` 的 `UDP socket` 通过 `sendto/recvfrom` 显式携带远端地址。

UDP 的收发使用 `block_on` 模式：先 `poll` 回环接口，再尝试 `send_slice` 或 `recv_slice`；如果 `smoltcp` 返回缓冲区暂不可用，阻塞 `socket` 会让出 CPU 后重试，非阻塞 `socket` 直接返回 `EAGAIN`。等待期间任务被标记为 `interruptible`，收到信号时返回 `EINTR`，这和第七章的睡眠、`futex` 等阻塞路径保持一致。

TCP 对应 `TcpSocket`，在 `smoltcp` TCP 状态机之外维护一组上层连接状态：`CLOSED`、`BUSY`、`CONNECTING`、`CONNECTED` 和 `LISTENING`。`connect` 从 `CLOSED` 进入 `CONNECTING`，调用 `smoltcp` 发送 SYN，并在阻塞模式下等待状态变为 `Established`；非阻塞模式下先返回 `EINPROGRESS`，后续通过 `poll` 可写状态观察连接完成。`send/recv` 只允许在 `CONNECTED` 状态执行，处理半关闭、队列为空、对端关闭和连接重置等情况时映射到 `EAGAIN`、`EPIPE`、`ENOTCONN`、`ECONNRESET` 等 Linux 风格错误码。

`connect` 的核心路径如下。它先确保 `smoltcp socket handle` 存在，再调用 `smoltcp connect` 发送 SYN；本地和远端端点由 `smoltcp` 返回后写入 `TcpSocket`。阻塞模式下，后续 `block_on` 会等待 `poll_connect` 观察到 `Established`。

```
let handle = self.handle.get().read()
    .unwrap_or_else(|| socket_set().lock().add(SocketSetWrapper::new_tcp_socket()));
self.reset_shutdown_flags();
let bound_endpoint = self.bound_endpoint();
let remote_ipendpoint = from_sockaddr_to_ipendpoint(remote_addr);

let (local_endpoint, remote_endpoint) = socket_set()
    .lock()
    .with_socket_mut::<_, tcp::Socket, Result<(IpEndpoint, IpEndpoint), Errno>>(
        handle,
        |socket| {
            socket.connect(iface.lock().context(), remote_ipendpoint, bound_endpoint)?;
            Ok((socket.local_endpoint().unwrap(), socket.remote_endpoint().unwrap()))
        },
    )?;
```

TCP/UDP 的阻塞语义都落在 `block_on` 循环中。它不是忙等：每轮先驱动回环接口，再检查信号和 `real timer`，只有底层仍返回 `EAGAIN` 时才让出 CPU。

```
let result = loop {
    poll_interfaces();
    task.check_real_timer();
    if task.check_signal_interrupt() || task.is_interrupted() {
        task.clear_interrupted();
        break Err(Errno::EINTR);
    }
    match f() {
        Ok(res) => break Ok(res),
        Err(Errno::EAGAIN) => yield_current_task(),
        Err(e) => break Err(e),
    }
};
```

监听 TCP socket 通过 `listen` 注册到 `LISTEN_TABLE`。每个端口保存一个当前 `listen handle` 和一个已完成握手的 `accept` 队列；`promote_listener` 发现当前 `listen handle` 已经进入 `Established` 后，会把它移入 `accept` 队列，并创建新的 `smoltcp socket` 继续监听同一端口。`accept` 从队列中取出已连接 `handle`，构造新的 `TcpSocket` 并返回给系统调用层分配 `fd`。

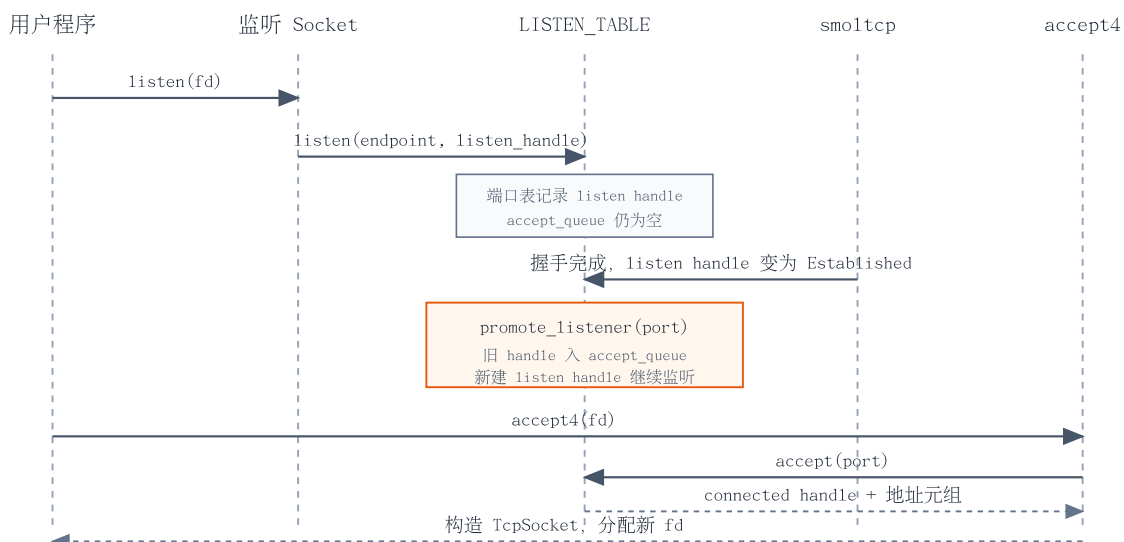


图 18 TCP listen / accept 序列

项目	TCP	UDP
协议对象	TcpSocket + smoltcp TCP handle	UdpSocket + smoltcp UDP handle
连接语义	connect 握手, listen/accept 管理入站连接	connect 只记录默认远端, 仍是数据报语义
阻塞等待	poll 接口、检查状态、EAGAIN 时 yield	poll 接口、检查缓冲区、EAGAIN 时 yield
地址状态	本地端点和远程端点随连接状态维护	本地端点由 bind/隐式 bind 设置, 远端端点可选
关闭语义	支持 shutdown 半关闭和 drop 时清理 smoltcp handle	shutdown 关闭 UDP socket 并在 drop 时移除 handle

表 17 TCP/UDP 实现对照

8.4 Port 端口分配

TCP 和 UDP 都使用 `0xc000` 到 `0xffff` 的临时端口区间, 并分别维护一个静态游标。分配时从当前游标开始循环扫描, 跳过已经被对应协议占用的端口; TCP 还会同时检查 `LISTEN_TABLE`, 避免临时端口和监听端口冲突。若用户显式绑定端口, 协议层会先通过 `tcp_bind_check` 或 `udp_bind_check` 检查全局 `SocketSet` 中是否已有同地址同端口 socket; 设置 `SO_REUSEADDR` 时则放宽这一检查。

`socket option` 的目标是兼容常见用户程序, 而不是完整复刻 Linux 网络栈。`SO_SNDBUF`、`SO_RCVBUF`、`SO_RCVTIMEO`、`SO_SNDTIMEO` 等选项主要保存在 `Socket` 对象中, 供查询和部分等待路径使用; `TCP_NODELAY` 会映射到 smoltcp 的 Nagle 开关; `IP_TTL` 会设置 TCP/UDP hop limit; `SO_ERROR` 当前返回 0, `TCP_MAXSEG` 返回默认 MSS。对于暂未支持的 `SOL_SOCKET/IP/TCP` 选项, 系统调用层按语义返回 `ENOPROTOPT` 或 `EOPNOTSUPP`。

本章小结

网络模块用 **Socket** 对象把 Linux socket ABI 接入 fd 表，再由 TCP/UDP 封装 smoltcp 协议状态机。当前实现聚焦 IPv4 回环通信，使用 **LoopbackDev** 主动轮询推进收发；TCP 通过监听表和 accept 队列支持本地连接，UDP 通过数据报 socket 支持 bind/connect/sendto/recvfrom，端口分配和常见 socket option 则补齐 libc 与 LTP 所需的兼容边界。

第九章

设备

9.1 设备管理模块概述

设备模块承担两类职责：一类是面向内核文件系统和存储路径的真实驱动抽象，例如 VirtIO block；另一类是面向用户态 Linux 兼容性的设备文件，例如 `/dev/null`、`/dev/zero`、`/dev/random`、`/dev/misc/rtc` 和 `loop` 设备。RespOS 当前没有实现通用设备模型、热插拔或完整设备树解析，而是以静态平台配置和少量 `trait` 抽象支撑初赛阶段需要的设备能力。

从层次上看，`drivers/` 提供内核内部设备 `trait` 和 VirtIO 块设备驱动；`fs/dev/` 提供 `devfs`，把特殊设备以 VFS inode 的形式暴露到 `/dev`；架构配置模块提供 QEMU virt 平台上的 MMIO、PCI、UART、RTC/测试设备等地址范围。这样的组织方式比较轻量，但边界清楚：块设备服务 Ext4 根文件系统，字符/伪设备服务用户态兼容性，平台配置服务内核高地址映射和具体 `transport` 创建。

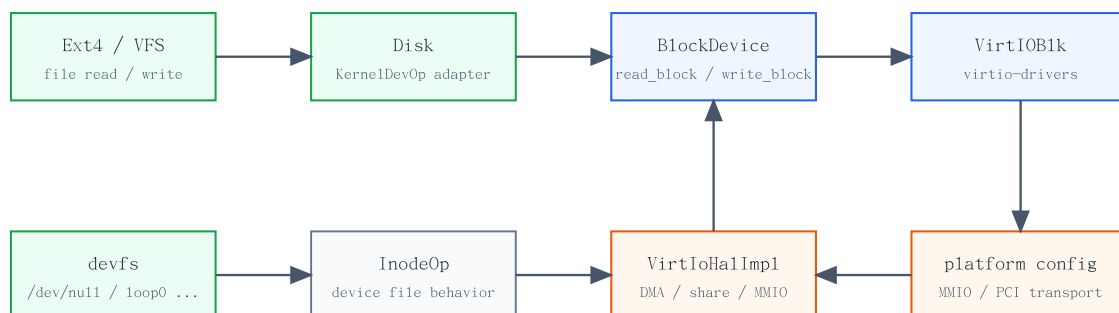


图 19 设备模块分层

9.2 设备抽象与块设备

`drivers/device.rs` 定义了统一设备分类和操作错误类型。当前真正被文件系统主路径使用的是 `BlockDevice` `trait`，它在 `Device` 基础上补充块数量、块大小、按块读写和 `flush` 能力。块设备驱动不直接暴露给用户态，而是被 `Disk` 适配层包装成 `lweext4` 需要的连续读写接口。

```

pub trait Device: Send + Sync {
    fn device_name(&self) -> &str;
    fn device_type(&self) -> DeviceType;
}

pub trait BlockDevice: Device {
    fn num_blocks(&self) -> usize;
    fn block_size(&self) -> usize;
    fn read_block(&self, block_id: usize, buf: &mut [u8]) -> DevResult;
}

```

```

fn write_block(&self, block_id: usize, buf: &[u8]) -> DevResult;
fn flush(&self) -> DevResult;
}

```

Disk 的作用是把“按偏移连续读写”转换成“按块读写”。当访问刚好落在块边界且长度不少于一个块时，它直接调用底层 `read_block` 或 `write_block`；当访问只覆盖部分块时，它会先读出完整块，修改局部数据，再写回完整块。这样 Ext4 后端可以通过 `lwext4` 的 `KernelDevOp` 接口使用块设备，而不需要在文件系统层手写块对齐逻辑。

9.3 VirtIO 块设备

RespOS 的根文件系统后端依赖 VirtIO block。RISC-V 64 路径使用 QEMU virt 平台的 MMIO transport：从 `VIRTIO_MMIO[0]` 取出基址和长度，加上 `KERNEL_BASE` 得到内核虚拟地址，再创建 `MmioTransport`。LoongArch 64 路径使用 PCI transport，通过平台 PCI 扫描找到 `virtio-blk` 设备。二者最终都构造同一个 `VirtIoBlkDev<VirtIoHalImpl, T>`，向上实现 `BlockDevice`。

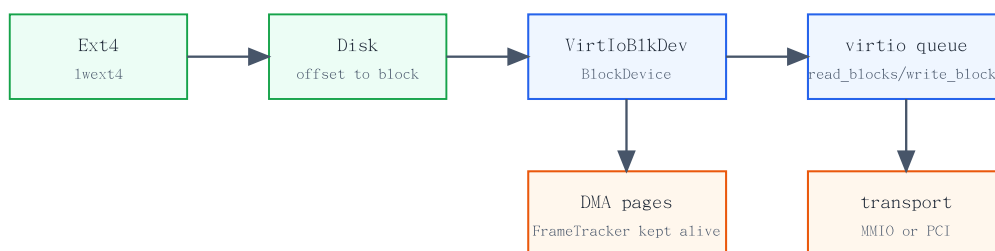


图 20 VirtIO block 到 Ext4 的数据路径

VirtIO 驱动的核心代码很薄，主要把 `virtio-drivers` 的 `VirtIOBlk` 包装成 RespOS 的设备接口，并把底层错误转换为 `DevError`。真正和内存管理相关的部分在 `VirtIoHalImpl`：DMA 分配通过页帧分配器取得连续页帧，用 `DMA_ALLOCATIONS` 保存 `FrameTracker`，直到 `virtio` 驱动调用 `dma_dealloc`；`share` 把内核虚拟地址转换为物理地址，供设备访问。

```

impl<H: Hal + 'static, T: Transport + 'static> BlockDevice for VirtIoBlkDev<H, T> {
    fn read_block(&self, block_id: usize, buf: &mut [u8]) -> DevResult {
        self.inner.lock().read_blocks(block_id as _, buf).map_err(as_dev_err)
    }

    fn write_block(&self, block_id: usize, buf: &[u8]) -> DevResult {
        self.inner.lock().write_blocks(block_id as _, buf).map_err(as_dev_err)
    }

    fn flush(&self) -> DevResult {
        self.inner.lock().flush().map_err(as_dev_err)
    }
}

```

需要注意的是，当前 DMA 分配假设页帧分配器能返回连续物理页，并用断言检查这一点。这符合初赛阶段单块设备和简单 I/O 压力下的实现取舍，但它也标记了后续改进方向：若要支持更复杂的 DMA 场景，需要页帧分配器提供明确的连续页分配能力，或在 HAL 层引入 `bounce buffer`。

9.4 devfs 设备文件

devfs 把设备对象纳入第五章介绍的 VFS 路径模型。初始化时，内核在根文件系统中创建 `/dev` 挂载点，将 `DevSuperBlock` 挂载到该目录，并把 devfs 根 dentry 固定到 dentry cache。用户程序随后可以用普通 `openat`、`read`、`write`、`stat`、`ioctl` 访问这些设备文件，系统调用层不需要为每个设备新增独立入口。

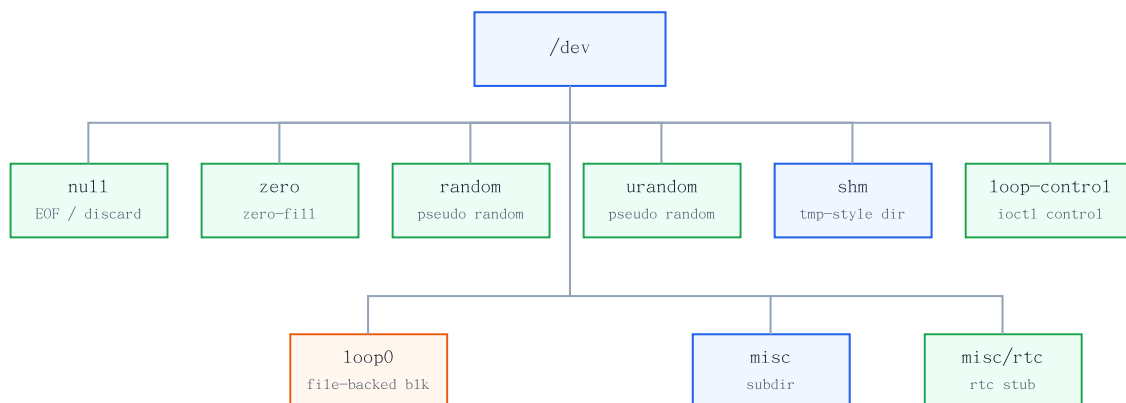


图 21 `/dev` 目录结构

devfs 根目录的 `lookup` 根据文件名返回不同 `inode`：`null`、`zero`、`random`、`urandom`、`shm`、`misc`、`loop-control` 和 `loop0`。这些 `inode` 直接实现 `InodeOp`，因此设备行为由读写函数决定。例如 `/dev/null` 读返回 0、写返回写入长度；`/dev/zero` 读时填充 0；`/dev/random` 和 `/dev/urandom` 用时间、偏移和常量混合生成伪随机字节；`/dev/misc/rtc` 当前是兼容性 `stub`，读返回 0、写入成功消费字节。

路径	类型	行为
<code>/dev/null</code>	字符设备	读取 EOF，写入丢弃并返回长度。
<code>/dev/zero</code>	字符设备	读取填充零字节，写入丢弃。
<code>/dev/random</code> / <code>/dev/urandom</code>	字符设备	返回基于时间和偏移混合的伪随机字节，写入成功消费。
<code>/dev/misc/rtc</code>	字符设备	当前作为 RTC 兼容入口，读取 0 字节，写入成功消费。
<code>/dev/loop-control</code>	字符设备	支持 <code>LOOP_CTL_GET_FREE</code> ，用于查询可用 loop 设备。
<code>/dev/loop0</code>	块设备	通过 <code>ioctl</code> 绑定普通文件作为后端，读写转发到底层 <code>FileOp</code> 。

表 18 devfs 主要设备语义

`/dev/loop0` 是 devfs 中相对复杂的设备。`LOOP_SET_FD` 会把当前任务指定 `fd` 对应的 `FileOp` 保存为 loop 后端；`LOOP_CLR_FD` 清除绑定；`BLKGETSIZE64` 和 `BLKGETSIZE` 根据后端文件大小返回块

设备大小。真正读写时，loop inode 会 seek 到指定偏移，再调用后端文件的 read 或 write。这让 mount、镜像处理和部分 LTP 测例可以把普通文件当作块设备使用。

```
match request {
  LOOP_SET_FD => {
    let task = current_task().expect("[kernel] current task is None.");
    let file = task.get_fd_entry(arg)?.file;
    *LOOP0_BACKEND.lock() = Some(file);
    Ok(0)
  }
  LOOP_CLR_FD => {
    if LOOP0_BACKEND.lock().take().is_some() { Ok(0) } else { Err(Errno::ENXIO) }
  }
  request if request & 0xffff == BLKGETSIZE64 & 0xffff => {
    let size = loop_backend()?.get_stat()?.size as u64;
    copy_to_user(arg as *mut u64, &size as *const u64, 1)?;
    Ok(0)
  }
  _ => Err(Errno::EINVAL),
}
```

9.5 控制台与平台设备配置

控制台路由 console.rs、架构 sbi.rs 和 fs/stdio.rs 共同完成。RISC-V 通过 SBI legacy console 接口输出和读取字符；LoongArch 因为直接运行在裸机环境中，使用 UART MMIO 寄存器轮询发送/接收。标准输入输出对象实现 FileOp，在任务创建时作为 fd 0、1、2 的基础对象，使用用户态程序可以通过普通 read/write 使用控制台。

平台设备配置目前以静态常量表达，而不是运行时解析完整设备树。RISC-V 的 VIRTIO_MMIO 记录 QEMU virt 上 virtio-mmio-bus 的地址区间；LoongArch 的 MMIO 记录 VirtIO block、UART、RTC/测试设备、PCI ECAM 和 PCI BAR window 等地址区间，并通过 pub use board::MMIO as VIRTIO_MMIO 兼容共享代码命名。内存管理启动阶段会把这些 MMIO 区域加入内核高地址映射，驱动层再基于这些地址创建 transport 或直接访问寄存器。

项目	RISC-V 64	LoongArch 64
块设备 transport	VirtIO MMIO，使用 VIRTIO_MMIO[0] 创建 MmioTransport。	PCI transport，通过 PCI 扫描找到 virtio-blk。
控制台	SBI legacy console。	UART MMIO，轮询 LSR/THR/RBR 寄存器。
MMIO 配置	0x1000_1000、0x1000_2000 两个 virtio-mmio 区间。	VirtIO、UART、RTC/测试设备、PCI ECAM、PCI BAR window 等静态区间。
关机/复位	SBI system reset。	ACPI GED sleep/reset MMIO 寄存器。

表 19 双架构平台设备配置

本章小结

设备章的关键是把硬件驱动、VFS 设备文件和平台地址配置分开处理。VirtIO block 通过 BlockDevice 和 Disk 适配层支撑 Ext4 根文件系统；devfs 通过 InodeOp 暴露 Linux 常见 /dev 入口；控制台和平台 MMIO/PCI 配置则由架构层提供，保证上层代码能以统一接口使用底层设备能力。

第十章

硬件抽象层

10.1 硬件抽象层总览

RespOS 同时支持 RISC-V 64 和 LoongArch 64。为了避免上层内核在系统调用、任务调度、文件系统、网络和设备路径中到处出现架构条件编译，项目把架构相关能力集中放在 `os/src/arch/` 下，再由 `arch/mod.rs` 根据编译目标统一导出。上层代码使用的是 `crate::arch::trap`、`crate::arch::timer`、`crate::arch::mm::PageTable`、`crate::arch::read_mmu_token`、`crate::arch::sfence` 等接口，具体寄存器、汇编入口和页表格式留给各架构目录实现。

这种组织方式不是传统意义上完整的 HAL trait 对象层，而是“同名模块 + 条件编译 + 统一 re-export”。它的优点是零运行时开销，缺点是两个架构必须维持相同的模块边界和函数签名。对当前内核来说，这个取舍比较合适：架构差异主要集中在启动、trap、上下文切换、时钟、中断开关、页表和平台配置，其他内核子系统可以保持共享。

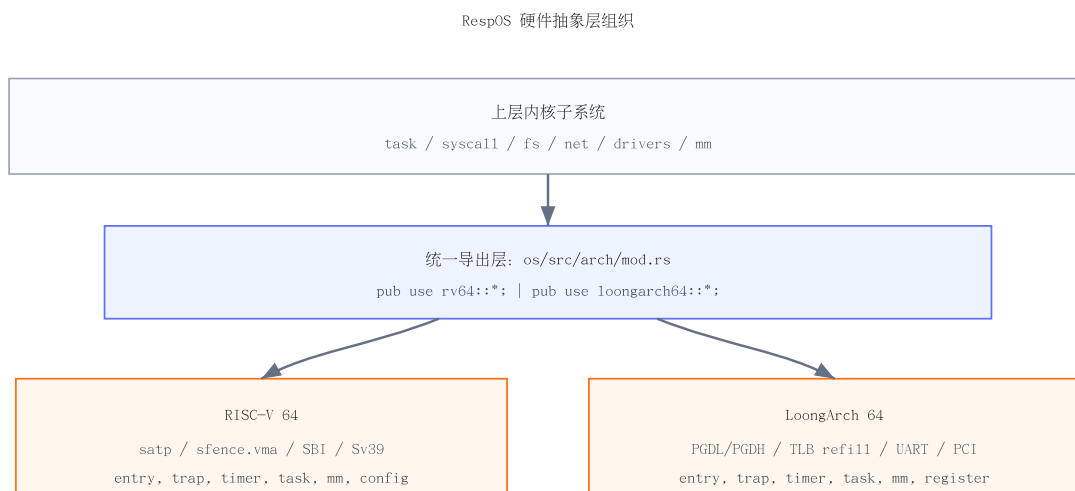


图 22 硬件抽象层组织

```

#[cfg(target_arch = "riscv64")]
pub mod rv64;
#[cfg(target_arch = "riscv64")]
pub use rv64::*;

#[cfg(target_arch = "loongarch64")]
pub mod loongarch64;
#[cfg(target_arch = "loongarch64")]
pub use loongarch64::*;
  
```

下表列出了第十章涉及的主要抽象边界。可以看到，RespOS 并没有强行把所有硬件差异塞进一个巨大的 trait，而是按照内核实际依赖拆成若干小接口。

接口	共享语义	架构差异
entry	从汇编入口进入 <code>rust_main</code> ，完成早期栈和高地址切换。	RISC-V 在汇编中写 <code>satp</code> ；LoongArch 先依赖 DMW，再在 Rust 中建立临时页表。
trap	保存用户上下文，分发系统调用、页错误和时钟中断。	寄存器、异常码、返回指令和系统调用参数寄存器不同。
timer	提供 <code>tick</code> 读取、下一次中断设置和时间尺度换算。	RISC-V 读 <code>time</code> CSR；LoongArch 用 <code>rdtime.d</code> 和板级频率配置。
mm	提供 <code>PageTable</code> 、 <code>PageTableEntry</code> 和 <code>PTEFlags</code> 。	RISC-V 使用 Sv39 PTE；LoongArch 使用 LA64 PTE、PGDL/PGDH 和 TLB refill。
task	暴露 <code>__switch</code> ，保存和恢复内核上下文。	汇编保存的寄存器集合和 ABI 名称不同。
config	给上层提供内存、设备、文件系统和 <code>syscall</code> 常量。	内存起点、MMIO/PCI/UART/SBI 等平台信息不同。

表 20 HAL 主要接口边界

10.2 处理器访问接口

处理器访问接口主要由 `read_mmu_token`、`write_mmu_token`、`sfence`、`idle`、中断守卫和 LoongArch 本地 CSR 封装组成。RISC-V 路径依赖 `riscv crate` 提供的寄存器接口；LoongArch 路径则在 `register/mod.rs` 中用 `csrrd`、`csrwr` 封装当前内核实际用到的 CSR，避免在业务代码中散落裸汇编。

MMU token 是上层地址空间切换最关键的处理器接口。RISC-V 的 token 直接对应 `satp`，写入后用 `sfence.vma` 刷新地址转换；LoongArch 的 token 是根页表物理地址，必须同时写入 PGDL 和 PGDH，因为硬件会按虚拟地址所在半区选择低半区或高半区根页表。RespOS 当前使用 ASID 0，因此切换地址空间后需要刷新 TLB，保证用户页表变化立即生效。

```
// RISC-V 64
pub fn read_mmu_token() -> usize {
    satp::read().bits()
}
pub fn write_mmu_token(token: usize) {
    satp::write(token);
}
pub fn sfence() {
    unsafe { asm!("sfence.vma", options(nostack)); }
}

// LoongArch 64
pub fn write_mmu_token(token: usize) {
    unsafe {
        register::mmu::write_pgdl(token);
        register::mmu::write_pgdh(token);
        register::mmu::write_asid(0);
        register::mmu::sync_page_table_root();
    }
}
```

中断守卫同样维持统一语义：构造时保存当前中断使能状态并关闭中断，析构时只在原本开启的情况下恢复。RISC-V 操作 `sstatus.SIE`，LoongArch 操作 `CRMD.IE`。这让锁、调度和临界区代码可以使用同一个 `InterruptGuard` 名称，不需要关心底层控制位位置。

10.3 内核入口例程

内核入口是两套架构差异最大的地方之一。RISC-V QEMU virt 由固件加载到 `0x8020_0000` 附近，`entry.asm` 设置早期栈后建立一个极简 `boot_pagetable`：一份低地址直接映射服务早期物理内存访问，另一份高半区线性映射服务内核虚拟地址。随后写入 `satp`、执行 `sfence.vma`，再由 `enter_main` 把栈指针和 `rust_main` 地址加上 `KERNEL_BASE`，跳转到高地址内核。

LoongArch QEMU virt 的启动约束不同。内核 ELF 链接在高半区，但 CPU 进入 `_start` 时仍处于直接地址模式。汇编入口先配置 `DMW0` 和 `DMW1`，保留低地址和当前执行段的直接映射；`rust_main` 中调用 `enable_boot_paging()` 构造临时三级页表，配置页表遍历和 TLB refill 入口，再通过 `jump_to_high_half()` 切到高半区。正式 `mm::init()` 激活 `KERNEL_SPACE` 后，还会关闭低地址 DMW，减少低地址别名带来的风险。

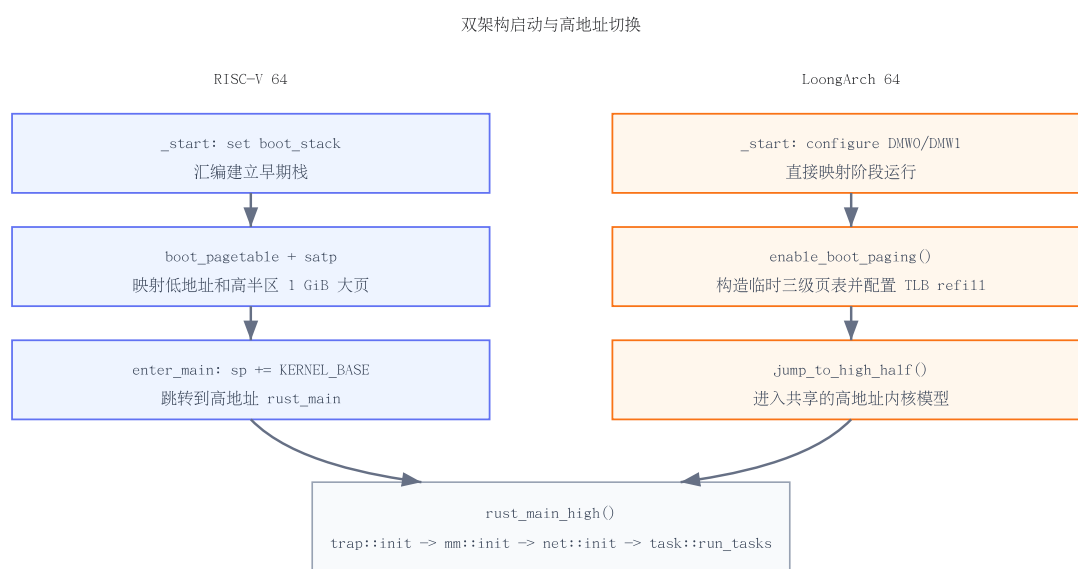


图 23 双架构启动流程

```

pub fn rust_main() -> ! {
    clear_bss();

    #[cfg(target_arch = "loongarch64")]
    {
        arch::enable_boot_paging();
        unsafe { arch::jump_to_high_half(rust_main_high as usize); }
    }

    #[cfg(target_arch = "riscv64")]
    rust_main_high()
}
  
```

进入 `rust_main_high()` 后，两套架构重新汇合：初始化 `trap`、堆和页帧分配器，激活内核地址空间，初始化网络，加入 `init` 进程，开启时钟中断并进入调度循环。也就是说，第十章的 HAL 设

计把“启动阶段的特殊性”尽量压缩到入口和 MMU 早期切换中，启动完成后的上层初始化顺序保持一致。

10.4 内存管理单元与地址空间

内存管理章节已经介绍了 `MemorySet`、`MapArea`、懒分配、`mmap` 和 `COW`。这里关注 HAL 视角下的 MMU 接入：共享代码负责“哪些虚拟区间应该映射到哪些物理页，以及权限是什么”；架构页表后端负责“如何把这个映射编码成硬件能理解的页表项，并如何激活它”。

RespOS 在两个架构上都采用 39-bit 虚拟地址、4 KiB 页和三级页表组织。`mm/address.rs` 提供统一的 `PhysAddr`、`VirtAddr`、`PhysPageNum`、`VirtPageNum` 类型；`arch/*/mm/page_table.rs` 各自提供 `PageTable` 和 `PageTableEntry`。这样 `MemorySet` 可以调用同名的 `map`、`unmap`、`translate`、`modify_pte`，而不需要知道底层 PTE 位布局。

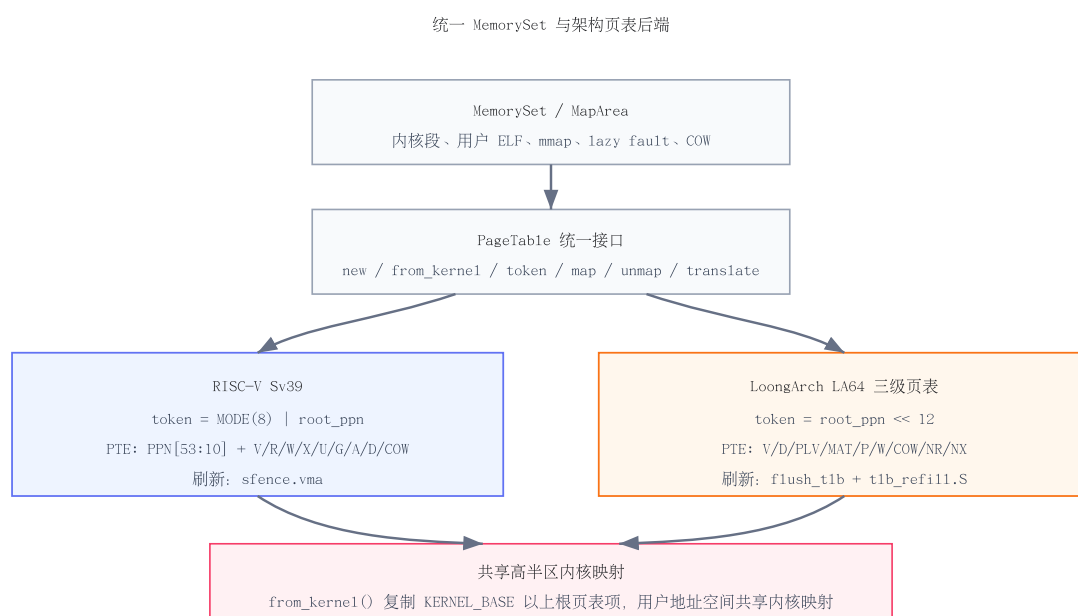


图 24 MemorySet 与架构页表后端

10.4.1 物理内存

物理内存范围由板级配置提供。RISC-V QEMU virt 的可用内存从 `0x8020_0000` 开始，当前配置到 `0x8800_0000`；LoongArch QEMU virt 在低地址放置 RAM，当前配置为 `0x0..0x0800_0000`。页帧分配器在 `mm::init()` 中初始化，随后内核堆、页表页、用户页、DMA 页和内核栈映射都通过页帧分配器获得物理页。

物理地址到内核虚拟地址的转换也有架构差异。RISC-V 路径始终使用 `pa + KERNEL_BASE` 的高半区线性映射；LoongArch 在分页尚未开启时可以直接访问低物理地址，分页开启后才使用 `pa + KERNEL_BASE`。这个细节被封装在 `PhysAddr::kernel_addr()` 里，上层通过 `ppn.get_bytes_array()` 或 `pa.get_mut()` 访问页内容时不需要分支。

```
#[cfg(target_arch = "loongarch64")]
fn kernel_addr(self) -> usize {
```

```

    if crate::arch::paging_enabled() {
        self.0 + KERNEL_BASE
    } else {
        self.0
    }
}

#[cfg(target_arch = "riscv64")]
fn kernel_addr(self) -> usize {
    self.0 + KERNEL_BASE
}

```

10.4.2 分页地址翻译模式

RISC-V 使用 Sv39。根页表物理页号编码在 `satp` 低位，`MODE=8` 表示 Sv39；页表项把 PPN 放在 bit 10 之后，权限位使用硬件定义的 V/R/W/X/U/G/A/D，RespOS 额外使用软件位保存 COW 标记。`PageTable::token()` 因此返回 `(8usize << 60) | root_ppn`。

LoongArch 也采用三级页表，但寄存器和 PTE 位语义不同。当前实现把根页表物理地址左移 12 位作为 token，写入 PGDL 和 PGDH；PTE 使用 V、D、PLV、MAT、G、软件 P/W/COW、NR、NX 等位。共享的 `PTEFlags::READ/WRITE/EXECUTE/USER/COW` 会在 LoongArch 后端转换成硬件 PTE bits，例如没有 READ 权限时设置 NR，没有 EXECUTE 权限时设置 NX。

项目	RISC-V 64	LoongArch 64
虚拟地址	39-bit Sv39。	39-bit LA64 三级页表。
根页表 token	<code>(8 << 60) root_ppn</code> ，写入 <code>satp</code> 。	<code>root_ppn << 12</code> ，写入 PGDL 和 PGDH。
PTE 权限	V/R/W/X/U/G/A/D，软件位保存 COW。	V/D/PLV/MAT/G/P/W/COW/NR/NX，由通用 flags 转换。
刷新方式	<code>sfence.vma</code> 。	<code>flush_tlb()</code> ，并依赖 <code>tlb_refill.S</code> 处理 TLB refill。
内核共享映射	<code>from_kernel()</code> 复制 <code>KERNEL_BASE</code> 以上根页表项。	<code>from_kernel()</code> 复制高半区 PGD 项。

表 21 页表模式对照

10.4.3 页表

两个架构的 `PageTable` 都维护根页表页帧和页表自身占用的 `FrameTracker` 列表。创建用户地址空间时，`PageTable::from_kernel()` 会分配新的根页表，并复制内核高半区对应的根页表项。这样每个用户进程拥有独立的低半区用户映射，同时共享同一套内核高半区映射，trap 进入内核后不需要再切换到单独的内核页表。

```

pub fn from_kernel() -> Self {
    let frame = frame_alloc().unwrap();
    let kernel_page_table = &KERNEL_SPACE.lock().page_table;
    let kernel_root_ppn = kernel_page_table.root_ppn;
    let index = VirtAddr::from(KERNEL_BASE).floor().indexes()[0];
    frame.ppn().get_pte_array()[index..]
        .copy_from_slice(&kernel_root_ppn.get_pte_array()[index..]);
}

```

```
PageTable { root_ppn: frame.ppn(), frames: vec![frame] }
}
```

页表的公共操作包括 `map`、`unmap`、`try_unmap`、`modify_pte`、`translate` 和 `translate_va`。共享的 `MemorySet` 在处理 ELF 装载、`mmap`、`brk`、`COW` 和 `page fault` 时只依赖这些操作。架构后端则负责找到三级页表中的目标 PTE，并在缺失中间页表页时分配新页。LoongArch 后端额外提供 `retire_owned_frames()`：当前所有用户进程使用 ASID 0，`release` 下短进程密集退出后立即回收页表页可能与残留地址转换状态冲突，因此页表页先进入有限隔离队列，超过上限后再释放旧页。

```
#[cfg(target_arch = "loongarch64")]
pub fn recycle_data_pages(&mut self) {
    self.areas.clear();
    self.page_table.retire_owned_frames();
}
```

10.4.4 直接映射窗口

直接映射窗口用于解决启动早期“还没有正式页表，但内核已经需要访问内存”的问题。RISC-V 的早期页表在汇编中建立，包含低地址直接映射和高半区线性映射；进入 `rust_main_high()` 时已经运行在高地址内核模型下，因此后续访问物理页统一走 `pa + KERNEL_BASE`。

LoongArch 则显式使用 `DMW.entry.asm` 先设置 `DMW0` 作为低地址直接映射，服务早期页表构建和低物理内存访问；同时设置 `DMW1` 保证当前执行段可访问。`enable_boot_paging()` 构造临时高地址页表后开启分页，并关闭 `DMW1`；正式内核页表激活后，`mm::init()` 调用 `disable_low_direct_map()` 关闭 `DMW0`，使内核回到更一致的高半区访问模型。

```
pub fn disable_low_direct_map() {
    unsafe {
        register::mmu::write_dmw0(0);
        register::mmu::flush_tlb();
    }
    LOW_DIRECT_MAP_ACTIVE.store(false, Ordering::Relaxed);
}
```

10.4.5 TLB 重填

RISC-V 的 Sv39 页表遍历由硬件完成，内核主要在切换页表或修改映射后执行 `sfence.vma`。LoongArch 的 QEMU virt 路径需要配置 TLB refill 入口，RespOS 在 `loongarch64/mod.rs` 中通过 `global_asm!(include_str!("tlb_refill.S"))` 引入 `__rfill`，启动时把该地址写入 `TLBREENTRY`，并配置页大小和页表遍历寄存器。

`tlb_refill.S` 的核心逻辑是从 PGD 出发，用 `lddir` 逐级查找目录项，再用 `ldpte` 加载相邻两个 PTE 并执行 `tlbfill`。如果中间目录缺失，就构造一个无效 TLB 项，使硬件随后转入普通页错误路径，由 Rust 层的 `trap_handler` 调用 `MemorySet::handle_page_fault` 完成懒分配、`COW` 或错误信号处理。

```
__rfill:
    csrwr    $t0, CSR_TLBSAVE
    csrrd    $t0, CSR_PGD
    lddir    $t0, $t0, 2
```

```
beqz    $t0, construct_invalid
addi.d  $t0, $t0, -1
lddir   $t0, $t0, 1
beqz    $t0, construct_invalid
addi.d  $t0, $t0, -1
ldpte   $t0, 0
ldpte   $t0, 1
tlbfill
```

trap 层把两套架构的页错误都归一成 `PageFaultCause::Instruction/Load/Store`。RISC-V 从 `scause` 和 `stval` 取异常类型与坏地址；LoongArch 从 `ESTAT` 和 `BADV` 取异常类型与坏地址。归一后进入同一个 `MemorySet::handle_page_fault`，因此上层懒分配、COW 和非法访问转 `SIGSEGV` 的策略可以复用。

本章小结

硬件抽象层把 RespOS 的双架构支持压缩在少数清晰边界内：`arch/mod.rs` 统一导出同名模块，启动和 trap 各自处理寄存器与 ABI 差异，页表后端把通用 `PTEFlags` 编码成不同硬件 PTE，`MemorySet` 则继续维护共享的地址空间语义。RISC-V 路径较直接，LoongArch 路径额外处理 DMW、PGDL/PGDH、TLB refill 和页表页延迟回收，这些差异都被封装在架构目录中，上层子系统可以保持一致代码路径。

第十一章

总结与展望

11.1 工作总结

初赛阶段，RespOS 的主要目标是把一个教学型 Rust 内核推进到能够承载 Linux ABI 兼容测例的状态。围绕这个目标，项目没有只停留在系统调用表的补项，而是补齐了用户程序运行所需的整条链路：任务创建与调度、trap 返回、虚拟内存、ELF 装载、VFS/Ext4、信号、IPC、时间、socket 回环网络、VirtIO block、devfs 以及双架构硬件抽象。现在的代码已经形成较清晰的模块边界，后续问题可以按“ABI 语义缺口、架构差异、资源生命周期、测试稳定性”几个方向继续定位。

从工程结构看，RespOS 已经具备三个基础能力。第一，内核主路径能够运行在 RISC-V 64 和 LoongArch 64 两套 QEMU virt 平台上，架构相关差异收敛在 `arch/` 目录。第二，文件、进程、内存、信号、时间和网络模块之间形成了统一的 Linux 风格错误码与 `fd/VFS` 接口，用户态程序可以通过普通系统调用组合使用这些能力。第三，项目建立了本地 LTP 日志过滤、报告生成和对比脚本，便于把一次测试失败追踪到具体 `syscall`、模块和平台差异。

方向	完成内容	支撑的用户态能力
任务与调度	实现 TCB、线程组、父子关系、FIFO 调度、 <code>fork/clone/execve/wait4</code> 和退出回收。	<code>shell</code> 、 <code>busybox</code> 、LTP 进程类测例可以创建、替换、等待和回收任务。
trap 与信号	双架构 <code>TrapContext</code> 、系统调用分发、页错误处理、信号递送和 <code>sigreturn</code> 返回路径。	用户态异常、安全系统调用入口、定时信号和阻塞等待中断。
内存管理	地址空间、三级页表、ELF 装载、用户栈/ <code>auxv</code> 、 <code>lazy fault</code> 、COW、 <code>brk</code> 、 <code>mmap</code> / <code>munmap</code> / <code>mprotect</code> 。	动态/静态 ELF、 <code>libc</code> 初始化、文件映射、匿名映射和进程复制。
文件系统	VFS、Ext4 后端、 <code>fd</code> 表、 <code>dentry cache</code> 、 <code>page cache</code> 、 <code>procfs</code> 、 <code>devfs</code> 和路径解析。	常规文件、目录、软链接、 <code>/proc</code> 、 <code>/dev</code> 、标准输入输出和文件描述符复制。
IPC 与同步	<code>pipe</code> 、FIFO、 <code>futex</code> 、部分 SysV IPC、 <code>robust list</code> 和线程退出同步。	<code>pthread</code> 、 <code>shell</code> 管道、进程间通信和 LTP 同步类测例。
时间与网络	多尺度时钟、 <code>interval timer</code> 、 <code>timerfd</code> 、IPv4 回环 TCP/UDP、 <code>socketpair</code> 和常见 <code>socket option</code> 。	<code>libc</code> 时间接口、超时等待、本机 <code>socket</code> 通信和网络相关测例。
设备与 HAL	VirtIO block、 <code>loop</code> 设备、控制台、RISC-V/LoongArch 启动、页表、TLB 和平台 MMIO/PCI 配置。	根文件系统访问、镜像挂载、双架构运行和平台设备访问。

表 22 初赛阶段主要完成内容

从测试角度看, 项目已经把测试流程从“手工观察串口输出”推进到“脚本化生成本地报告”。`judge/local-report/` 中保存了 RISC-V 与 LoongArch、glibc 与 musl 组合下的 LTP 报告; `judge/local-compare/` 可以和 Linux baseline 对比, 帮助区分内核语义问题、libc 差异和测例性价比问题。下表记录的是当前仓库中本地报告的 **TOTAL** 行统计, 它反映的是阶段性回归状态, 不代表最终比赛排名。

平台/ libc	TPASS	TFAIL	TBROK	TCONF	WARN	总断言
RISC-V + glibc	108	30	15	3	53	209
RISC-V + musl	110	39	24	6	1466	1645
LoongArch + glibc	116	37	21	4	4	182
LoongArch + musl	111	40	20	6	4	181

表 23 本地 LTP 报告阶段性统计

这些数字后面还需要结合测例列表和日志进一步解释: 有些失败来自尚未实现的边界语义, 有些来自 libc 行为差异, 有些则是耗时长但收益低的测例被主动跳过。文档前面的“初赛进展与排名”保留最终截图 TODO, 本章只总结当前工程和本地验证已经覆盖的范围。

总体来看, RespOS 初赛阶段最大的收获不是某一个单点功能, 而是建立了可以持续扩展的内核骨架。后续新增系统调用或修复测例时, 通常能落到明确的位置: 用户指针与错误码在 `syscall` 层处理, 资源对象通过 `fd/VFS/TCB/MemorySet` 管理, 架构差异由 `arch` 层兜底, 测试结果由本地报告 and 对比脚本回归。

11.2 未来计划

下一阶段工作应优先围绕“稳定性、兼容性、可验证性”推进, 而不是盲目扩张模块数量。当前内核已经覆盖了初赛常见主路径, 但大量 Linux ABI 细节仍然隐藏在错误码、权限检查、并发退出、信号时机和文件系统边界中。继续提升测例通过率时, 需要把这些问题拆成可复现、可回归的小任务。

方向	计划内容	预期收益
ABI 细节补齐	继续完善 <code>openat</code> 、权限检查、 <code>xattr</code> 、 <code>mount</code> 、 <code>socket option</code> 、 <code>SysV IPC</code> 和资源限制等边界语义。	减少 LTP 中因错误码、标志位和特殊路径不一致导致的失败。
文件系统稳定性	加强 <code>dentry/page cache</code> 一致性、 <code>Ext4</code> 写回路径、目录项修改、软链接和路径解析边界测试。	提升文件类、目录类和脚本类测例稳定性，降低长测例中的偶发错误。
内存与进程生命周期	继续审查 <code>fork/clone/execve/exit/wait4</code> 与 <code>COW</code> 、 <code>mmap</code> 、 <code>futex</code> 、 <code>robust list</code> 的交互。	减少短进程密集创建、线程退出和共享内存场景中的状态残留。
双架构一致性	把 RISC-V 与 LoongArch 的 <code>trap</code> 、时钟、页表、TLB、用户 ABI 和设备路径做成更系统的对照回归。	避免一个平台修复后另一个平台退化，提高双架构提交质量。
网络与设备扩展	在当前回环网络基础上评估 <code>VirtIO-net</code> ；完善 <code>loop</code> 、 <code>RTC</code> 、 <code>random</code> 、 <code>tty</code> 等设备文件语义。	支持更完整的用户态工具链和网络/设备类测例。
测试基础设施	将报告生成、baseline 对比、失败分类和耗时统计纳入固定流程，并保留关键失败日志。	让每次修改都能快速判断收益和回归风险。
文档维护	保持设计文档和代码同步；将开源项目借鉴、AI 协作、许可证和引用说明集中放在第十二章。	减少章节重复，使评审能从文档直接对应到实现位置。

表 24 后续优化计划

后续实现时还需要控制复杂度。对于初赛阶段尚不稳定的能力，应优先选择能解释、能回归的简单设计；只有当重复代码或状态分支已经影响维护时，再抽象成更通用的组件。例如调度策略可以暂时保持 FIFO，但阻塞唤醒和退出回收必须严格；网络可以先保持回环设备，但 `socket` 错误码、非阻塞语义和 `poll` 状态需要继续贴近 Linux；设备模型可以暂不追求热插拔，但 `devfs` 路径和 `ioctl` 兼容性需要服务真实用户程序。

本章小结

RespOS 初赛阶段已经形成可运行、可测试、可继续扩展的双架构宏内核主体。当前最重要的后续工作，是在已有模块边界内继续收敛 Linux ABI 细节、提升双架构一致性，并把本地 LTP 报告转化为稳定的回归流程。开源项目借鉴、AI 协作和许可证说明不在本章展开，统一放入第十二章。

第十二章

AI 协作与开源项目借鉴

- 12.1 AI 辅助完成的工作
- 12.2 人工审核与边界控制
- 12.3 借鉴的开源项目
- 12.4 许可证与引用说明